PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

GCode: Global Mapping and Analysis of Competitive Programming Activity

Submitted by:

PES1UG24AM123	Jayanth K	Sec C
PES1UG24AM162	Mohammad Bilal A	Sec C

Class of Prof. Raghu BA
Rati D Joshi

Abstract

Competitive programming platforms such as Codeforces receive thousands of submissions every hour, yet the activity is normally seen only per user or per problem. **GCode** turns this activity into an interactive 3D globe (styled after Radio Garden) on which every green dot is a city where programmers are active. Clicking a city opens a live panel with its submissions, users, acceptance rate, most practised topics, verdict mix, programming languages, difficulty bands and most solved problems.

The system combines a relational database and a document database, each used for the kind of data it suits. **MySQL** stores the normalized data — platforms, countries, cities, Codeforces accounts, contests, problems, tags, verdicts and submissions — with primary, foreign, unique and CHECK constraints, views, triggers, stored procedures, a full-text index and two database roles for role-based access control. **MongoDB** keeps the raw Codeforces API documents and builds one precomputed analytics document per city with aggregation pipelines ($lookup, $unwind, $group, $merge).

A background worker keeps the data live: every 5 seconds it polls the Codeforces API, writes new submissions to MySQL in a single transaction (TESTING submissions are later UPDATEd to their final verdict), stores the raw JSON in an outbox table, copies it to MongoDB every 15 seconds, geocodes new cities and rebuilds the city analytics every 5 minutes. Duplicates are prevented by unique keys in both databases. The web application is built with Next.js (React) and its API routes, with bcrypt-based login, signed session cookies and two roles (Admin and Viewer). During the test run reported here the system held 1,708 submissions from 1,140 accounts, 11,603 problems and 138 cities on the globe.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	7
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	9
5.	SQL DDL Statements	10
6.	SQL DML Statements	17
7.	NoSQL/ Vector Database Commands	28
8.	Results a) Resulting Tables' Screenshots	33
9.	Next.js-Project Tree Structure	40
10.	Few sample Screenshots of application code and demo	41
11.	Evaluation Rubrics- Completion Status	49

Introduction

1. Problem Statement

Codeforces, LeetCode, CodeChef and AtCoder have large communities of programmers, but their data is presented as rankings, user profiles and problem lists. It is hard to answer simple geographic questions such as **where in the world are people practising right now, which topics does each city focus on, and which problems are popular in a region?** The raw data needed to answer them is spread across several API endpoints, changes every few seconds, mixes final and still-judging verdicts, and only some users state their city.

The goal of GCode is to collect this data continuously, store it with strong integrity guarantees, analyse it per city, and present it on an interactive globe — without duplicate records and without slowing the user interface down with heavy queries on every click.

Short Description

GCode is a full-stack web application built as a proof of concept on Codeforces data, designed so that other platforms can be added later (every account, contest, problem and submission carries a `platform_id`).

- **Data collection:** a rate-limited Codeforces client (one request every 2.1 s) fetches recent submissions, user profiles, problems and contests; city names are converted to coordinates with OpenStreetMap Nominatim.
- **MySQL (relational core):** 17 tables in 3NF with constraints, views, triggers, stored procedures, transactions, a full-text index and database roles.
- **MongoDB (documents + analytics):** raw API documents and the `city_analytics` collection, rebuilt by aggregation pipelines and swapped in atomically.
- **Live pipeline:** a worker process polls every 5 s, syncs MongoDB every 15 s and rebuilds analytics every 5 min.
- **Next.js web app:** 3D globe with clickable cities, problems search, login, and an admin dashboard that shows the health of every pipeline stage.

User Requirements Specification

Users and Roles

Role	Who	Can do
Visitor (not logged in)	Anyone	Explore the globe, open city panels, search and filter problems
Viewer (`USER`)	Registered user	Everything a visitor can; blocked from admin pages and admin APIs (403)
Admin (`ADMIN`)	Project team	Admin dashboard: live totals, pipeline health, job history, rebuild analytics on demand

Functional Requirements

- FR1 – Show a 3D globe with one dot per city that has Codeforces activity; dot size grows with the number of submissions.
- FR2 – On clicking a city (or searching for it), show submissions, distinct users, acceptance %, top topics, verdicts, languages, difficulty bands and most solved problems.
- FR3 – Keep the data current: poll Codeforces every 5 s; new cities appear within minutes; the panel refreshes while open.
- FR4 – Store a submission once only; when a TESTING submission receives its final verdict, update the same record.
- FR5 – Problems page: full-text search by name, filter by tag and rating range, sorted by number of solvers.
- FR6 – Login with email and password (bcrypt); two roles, Admin and Viewer, enforced in the backend and in MySQL.
- FR7 – Admin dashboard: totals, submissions in the last 5 minutes, health of each pipeline stage, job log, manual rebuild.
- FR8 – Resolve free-text city names to coordinates, caching every attempt so the geocoder is never asked twice.

Non-functional Requirements

- Integrity enforced by the database (PK/FK/UNIQUE/CHECK constraints, transactions, triggers), not only by application code.
- Respect API rate limits (Codeforces ≈ 1 request / 2 s, Nominatim 1 request / s).
- A city click must be a single indexed read: analytics are precomputed, never calculated on demand.
- Credentials only in environment variables (`.env.local`, git-ignored); passwords stored only as bcrypt hashes.

Tech Stack and Justification

Layer	Technology	Why it fits this problem
Frontend	Next.js 15 (React 19), react-globe.gl (Three.js / WebGL)	Interactive 3D globe with clickable points; React state drives the side panel; required framework of the course.
Backend / API	Next.js Route Handlers (Node.js)	Database access stays on the server; one codebase for UI and API; credentials never reach the browser.
Relational DB	MySQL 8.0	Accounts, problems, submissions and geography are highly related and need FK/UNIQUE/CHECK constraints, transactions, triggers, procedures, views and full-text search.
Document DB	MongoDB 8.0	Stores the nested Codeforces JSON as-is and computes per-city analytics with aggregation pipelines.
Live sync	Node.js worker	Polls Codeforces, writes MySQL + outbox, syncs MongoDB, geocodes cities, rebuilds analytics on timers.
Security	bcryptjs, HMAC-signed httpOnly cookie, MySQL roles	Password hashing, tamper-proof sessions, and least-privilege database accounts per role.
Data sources	Codeforces API, OpenStreetMap Nominatim	Real, public competitive-programming data and free geocoding.

Why MongoDB as the NoSQL Database

- **The source data is JSON documents.** A Codeforces submission embeds the problem (with a tags array) and an author object with a members array; MongoDB stores it unchanged in `raw_submissions`, so nothing is lost and fields Codeforces adds later need no ALTER TABLE.
- **Analytics are document-shaped.** Each city's panel is one nested document (totals, topics[], verdicts[], languages[], difficulty[], mostSolved[]). It is built by aggregation pipelines and read with a single `findOne` by `cityId` — the globe never runs heavy joins when a user clicks.
- **Graph or vector databases do not fit.** There is no deep relationship traversal (Neo4j) and no semantic text search (vector DB) in this problem; key-value stores cannot aggregate.
- **Division of work:** MySQL is the source of truth with relational integrity; MongoDB holds raw documents and derived analytics. The outbox pattern copies each new or changed record from MySQL to MongoDB with idempotent upserts.

E-R Model

- **List of Entities (weak entity highlighted)**

Entity	Type	Key	Description
COUNTRY	Strong	country_id	A country (name, ISO-2 code)
CITY	Strong	city_id	A city with latitude/longitude — one dot on the globe
CITY_ALIAS	Strong	alias_id	Spellings of a city in user profiles ('Banga-lore', 'Bengaluru')
CITY_DAILY_STATS	WEAK (owner: CITY)	partial key stat_date	Submissions and accepted submissions per city per day
PLATFORM	Strong	platform_id	Codeforces now; LeetCode / CodeChef later
PLATFORM_AC-COUNT	Strong	account_id	A Codeforces handle with its raw and re-solved location and rating
APP_USER	Strong	user_id	A person who logs in to GCode (email, bcrypt hash, role)
CONTEST	Strong	contest_id	A Codeforces contest
PROBLEM	Strong	problem_id	A problem (external key e.g. 2060A, name, rating)
TAG	Strong	tag_id	A topic such as greedy, dp, graphs
SUBMISSION	Strong	submission_id	One attempt by an account on a problem, with language and time
VERDICT	Strong	verdict_id	OK, WRONG_ANSWER, TESTING … and whether it counts as accepted
GEOCODE_CACHE	Strong	(raw_city, raw_coun-try)	Every place name sent to the geocoder, found or not
INGEST_OUTBOX	Strong (standalone)	outbox_id	Raw API JSON waiting to be copied to MongoDB
SYNC_RUN	Strong (standalone)	run_id	One row per background job run (admin dashboard)

Relationships: contains (COUNTRY 1:N CITY), known_as (CITY 1:N CITY_ALIAS), scoped_to (COUNTRY 1:N CITY_ALIAS), records (identifying, CITY 1:N CITY_DAILY_STATS), resolves_to (CITY 1:N GEOCODE_CACHE), located_in (CITY 1:N PLATFORM_ACCOUNT), country_of (COUNTRY 1:N PLATFORM_ACCOUNT), hosts (PLATFORM 1:N PLATFORM_ACCOUNT), linked_to (APP_USER 1:N PLATFORM_ACCOUNT), follows (APP_USER M:N CITY, attribute followed_at), organizes (PLATFORM 1:N CONTEST), lists (PLATFORM 1:N PROBLEM), has_problem (CONTEST 1:N PROBLEM), has_tag (PROBLEM M:N TAG), makes (PLATFORM_ACCOUNT 1:N SUBMISSION), attempted_in (PROBLEM 1:N SUBMISSION), has_verdict (VERDICT 1:N SUBMISSION).

Non-simple Attributes

- **Composite key:** GEOCODE_CACHE (raw_city, raw_country); CITY_DAILY_STATS (city_id, stat_date) — weak entity, partial key stat_date.
- **Derived attributes:** CITY_ALIAS.country_key (generated column `IFNULL(country_id, 0)`, makes the UNIQUE key work when the country is unknown); per-city accepted % in city_analytics (accepted / submissions).

- **Multivalued attribute:** a problem's tags — modelled as the M:N relationship has_tag (relation problem_tag) instead of a list column.
- **Relationship attribute:** followed_at on the follows relationship (APP_USER – CITY).
- **Enumerated / restricted attributes:** PLATFORM_ACCOUNT.location_status (UNKNOWN, COUNTRY_ONLY, CITY_RESOLVED, UNRESOLVED), APP_USER.role (ADMIN, USER), SYNC_RUN.job and status.

- **ER Diagram**

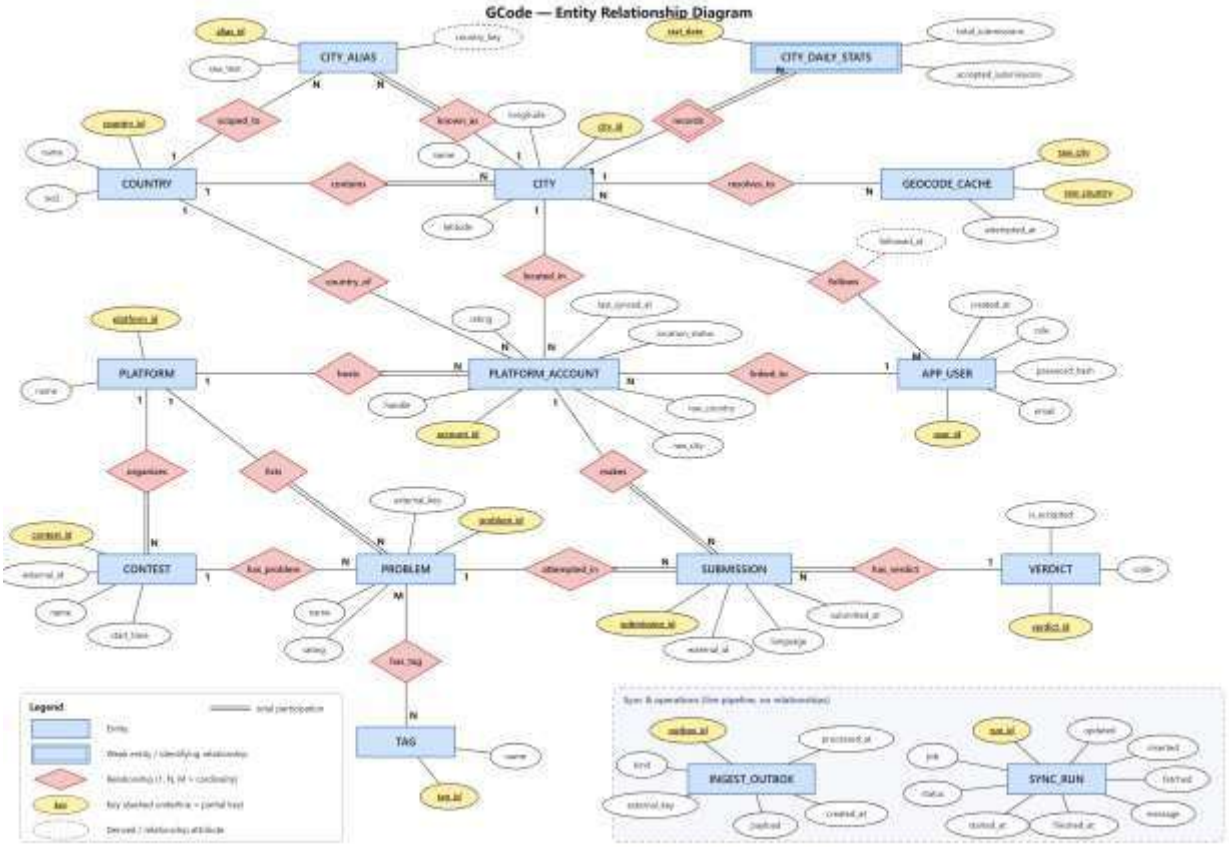


Figure 1: ER diagram of GCode (Chen notation; double border = weak entity / identifying relationship, double line = total participation)

Relational Model

- **List of Relations**

Relation	Primary key	Foreign keys	Origin in ER model
platform	platform_id	—	Entity
country	country_id	—	Entity
city	city_id	country_id → country	Entity
city_alias	alias_id	city_id → city, country_id → country	Entity
app_user	user_id	—	Entity
platform_account	account_id	platform_id, app_user_id, country_id, (city_id, country_id) → city	Entity
contest	contest_id	platform_id → platform	Entity
problem	problem_id	platform_id, contest_id → contest	Entity
tag	tag_id	—	Entity
problem_tag	(problem_id, tag_id)	problem_id → problem, tag_id → tag	NOT an entity: M:N relationship has_tag
verdict	verdict_id	—	Entity
submission	submission_id	platform_id, account_id, problem_id, verdict_id	Entity
city_daily_stats	(city_id, stat_date)	city_id → city	Weak entity (derived rollup)
city_follow	(user_id, city_id)	user_id → app_user, city_id → city	NOT an entity: M:N relationship follows
geocode_cache	(raw_city, raw_country)	city_id → city	Entity
ingest_outbox	outbox_id	—	Entity (live sync)
sync_run	run_id	—	Entity (live sync)

Highlighted rows are relations that are not entities in the ER model: they implement M:N relationships. All relations are in 3NF; `submission.platform_id` is a deliberate, trigger-checked denormalization that lets UNIQUE (platform_id, external_id) reject duplicate submissions.

- **Schema Diagram**

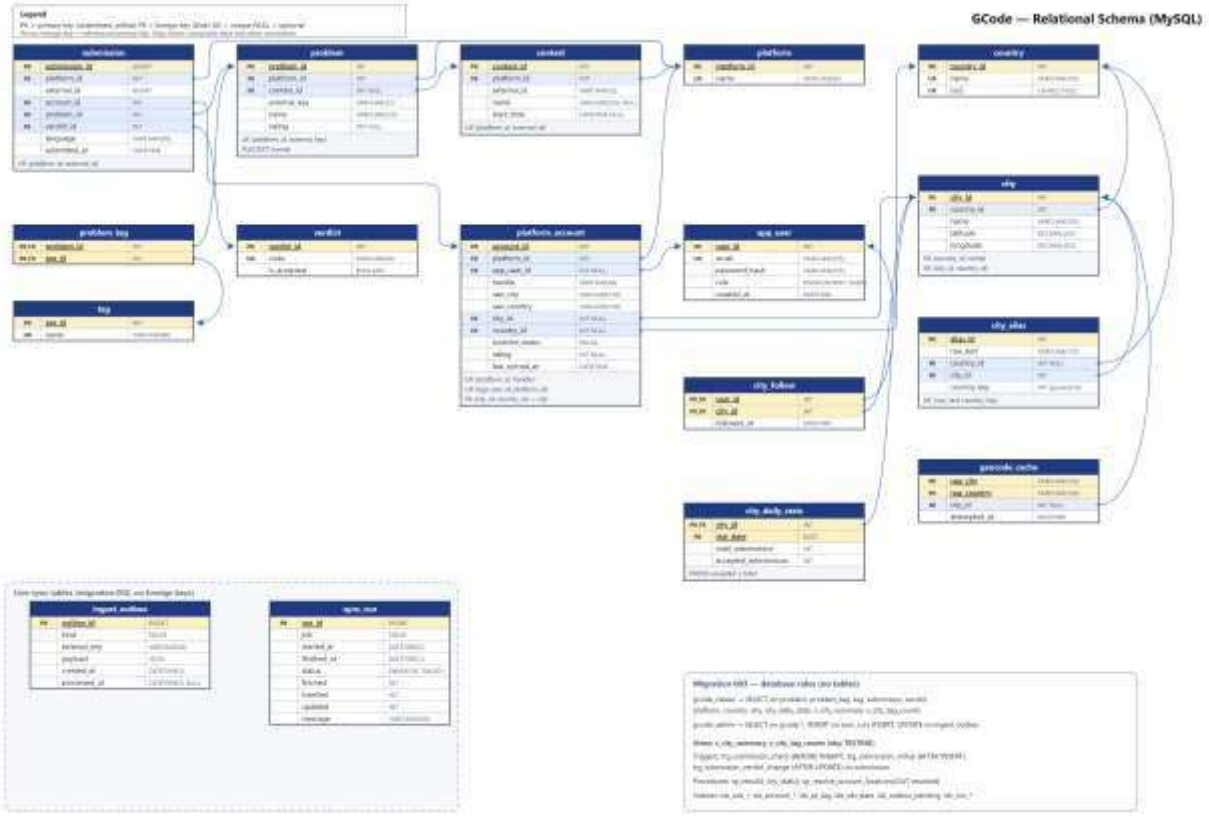


Figure 2: Relational schema (MySQL): keys, types, composite constraints and every foreign key

SQL DDL Statements

The complete DDL is in the repository (`db/schema.sql`, `db/migrations/002_live_sync.sql`, `db/migrations/003_roles.sql`). It uses CREATE DATABASE/TABLE/VIEW/TRIGGER/PROCEDURE/ROLE/USER, CREATE OR REPLACE VIEW, DROP TRIGGER/PROCEDURE IF EXISTS, GRANT, primary, foreign, unique, CHECK and FULLTEXT keys, a generated column and a JSON column. The listings below are the exact files.

db/schema.sql — tables, constraints, triggers, procedure, views

```
-- GCode: MySQL 8.0.16+ schema (CHECK constraints are enforced from 8.0.16)
--
-- Run with the mysql client:    mysql -u root -p < schema.sql
--   (DELIMITER is a client command, so Node drivers such as mysql2 cannot run this file directly)
-- Ingestion code must run    SET time_zone = '+00:00';    so DATE(submitted_at) buckets are UTC
--   (Codeforces sends epoch seconds and FROM_UNIXTIME follows the session time zone).
CREATE DATABASE IF NOT EXISTS gcode CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE gcode;
-- ---------------------------------------------------------------------------------------------- platforms
CREATE TABLE platform (
```

```
  platform_id INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(50) NOT NULL UNIQUE          -- 'Codeforces', 'LeetCode', ...
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------------------------------------- geography
CREATE TABLE country (
  country_id INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL UNIQUE,
  iso2       CHAR(2) NULL UNIQUE                  -- helps map to globe/geojson
) ENGINE=InnoDB;

CREATE TABLE city (
  city_id    INT AUTO_INCREMENT PRIMARY KEY,
  country_id INT NOT NULL,
  name       VARCHAR(100) NOT NULL,
  latitude   DECIMAL(9,6) NOT NULL,
  longitude  DECIMAL(9,6) NOT NULL,
  UNIQUE KEY uq_city_country_name (country_id, name),   -- 'Hyderabad' in India != Pakistan
  UNIQUE KEY uq_city_country (city_id, country_id),     -- target for composite FK below
  CONSTRAINT fk_city_country FOREIGN KEY (country_id) REFERENCES country(country_id),
  CONSTRAINT chk_city_lat CHECK (latitude  BETWEEN -90  AND 90),
  CONSTRAINT chk_city_lng CHECK (longitude BETWEEN -180 AND 180)
) ENGINE=InnoDB;

-- maps messy profile text ('Bengaluru', 'Bangalore', 'blr') to one canonical city
CREATE TABLE city_alias (
  alias_id    INT AUTO_INCREMENT PRIMARY KEY,
  raw_text    VARCHAR(150) NOT NULL,
  country_id  INT NULL,                                 -- optional disambiguation
  city_id     INT NOT NULL,
  country_key INT AS (IFNULL(country_id, 0)) STORED,    -- makes UNIQUE work with NULL country
  UNIQUE KEY uq_alias (raw_text, country_key),
  CONSTRAINT fk_alias_city    FOREIGN KEY (city_id)    REFERENCES city(city_id) ON DELETE CASCADE,
  CONSTRAINT fk_alias_country FOREIGN KEY (country_id) REFERENCES country(country_id)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------------------------------------- app users
-- people who log in to GCode (not the same thing as a Codeforces account)
CREATE TABLE app_user (
  user_id       INT AUTO_INCREMENT PRIMARY KEY,
  email         VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role          ENUM('ADMIN','USER') NOT NULL DEFAULT 'USER',
  created_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------------------------------------- accounts
CREATE TABLE platform_account (
  account_id      INT AUTO_INCREMENT PRIMARY KEY,
  platform_id     INT NOT NULL,
  app_user_id     INT NULL,                 -- the GCode user who linked this account (one user -> many accounts)
  handle          VARCHAR(64) NOT NULL,
  raw_city        VARCHAR(150) NULL,        -- exactly what the API returned, so we can re-resolve later
  raw_country     VARCHAR(100) NULL,
  city_id         INT NULL,
  country_id      INT NULL,
  location_status ENUM('UNKNOWN','COUNTRY_ONLY','CITY_RESOLVED','UNRESOLVED') NOT NULL DEFAULT 'UNKNOWN',
  rating          INT NULL,
  last_synced_at  DATETIME NULL,
  UNIQUE KEY uq_account (platform_id, handle),
  UNIQUE KEY uq_user_platform (app_user_id, platform_id),  -- a user links at most one account per platform (NULLs
repeat freely)
  KEY idx_account_city (city_id),
  KEY idx_account_country (country_id),
  CONSTRAINT fk_account_platform FOREIGN KEY (platform_id) REFERENCES platform(platform_id),
  CONSTRAINT fk_account_user     FOREIGN KEY (app_user_id) REFERENCES app_user(user_id) ON DELETE SET NULL,
  CONSTRAINT fk_account_country  FOREIGN KEY (country_id)  REFERENCES country(country_id),
  -- a city must belong to the account's country (skipped by MySQL when city_id is NULL)
  CONSTRAINT fk_account_city     FOREIGN KEY (city_id, country_id) REFERENCES city(city_id, country_id),
  -- location_status must agree with the id columns (also implies: a city always has a country)
  CONSTRAINT chk_account_status  CHECK (
       (location_status = 'CITY_RESOLVED' AND city_id IS NOT NULL)
    OR (location_status = 'COUNTRY_ONLY' AND city_id IS NULL AND country_id IS NOT NULL)
    OR (location_status = 'UNKNOWN'      AND city_id IS NULL AND country_id IS NULL)
    OR (location_status = 'UNRESOLVED'   AND city_id IS NULL)
```

```
  )
) ENGINE=InnoDB;

---
CREATE TABLE contest (
  contest_id  INT AUTO_INCREMENT PRIMARY KEY,
  platform_id INT NOT NULL,
  external_id VARCHAR(32) NOT NULL,
  name        VARCHAR(255) NULL,            -- NULL until contest.list is loaded: a submission's problem carries
only the contest id
  start_time  DATETIME NULL,
  UNIQUE KEY uq_contest (platform_id, external_id),
  CONSTRAINT fk_contest_platform FOREIGN KEY (platform_id) REFERENCES platform(platform_id)
) ENGINE=InnoDB;

CREATE TABLE problem (
  problem_id    INT AUTO_INCREMENT PRIMARY KEY,
  platform_id   INT NOT NULL,
  contest_id    INT NULL,                     -- not every platform has contests
  external_key VARCHAR(32) NOT NULL,          -- e.g. '2060A'
  name          VARCHAR(255) NOT NULL,
  rating        INT NULL,
  UNIQUE KEY uq_problem (platform_id, external_key),
  FULLTEXT KEY ft_problem_name (name),         -- full-text search requirement
  CONSTRAINT fk_problem_platform FOREIGN KEY (platform_id) REFERENCES platform(platform_id),
  CONSTRAINT fk_problem_contest  FOREIGN KEY (contest_id)  REFERENCES contest(contest_id),
  CONSTRAINT chk_problem_rating  CHECK (rating IS NULL OR rating BETWEEN 0 AND 4000)
) ENGINE=InnoDB;

CREATE TABLE tag (
  tag_id INT AUTO_INCREMENT PRIMARY KEY,
  name   VARCHAR(80) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE problem_tag (
  problem_id INT NOT NULL,
  tag_id     INT NOT NULL,
  PRIMARY KEY (problem_id, tag_id),
  KEY idx_pt_tag (tag_id),
  CONSTRAINT fk_pt_problem FOREIGN KEY (problem_id) REFERENCES problem(problem_id) ON DELETE CASCADE,
  CONSTRAINT fk_pt_tag     FOREIGN KEY (tag_id)     REFERENCES tag(tag_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-----------------------
```

```
  KEY idx_sub_time (submitted_at),
  CONSTRAINT fk_sub_platform FOREIGN KEY (platform_id) REFERENCES platform(platform_id),
  CONSTRAINT fk_sub_account  FOREIGN KEY (account_id)  REFERENCES platform_account(account_id),
  CONSTRAINT fk_sub_problem  FOREIGN KEY (problem_id)  REFERENCES problem(problem_id),
  CONSTRAINT fk_sub_verdict  FOREIGN KEY (verdict_id)  REFERENCES verdict(verdict_id)
) ENGINE=InnoDB;

-- derived table (so it is not in the ER diagram): pre-aggregated counts per city per day
-- feeds the globe + time slider cheaply; kept current by trg_submission_rollup, rebuilt by sp_rebuild_city_stats
CREATE TABLE city_daily_stats (
  city_id              INT NOT NULL,
  stat_date            DATE NOT NULL,
  total_submissions    INT NOT NULL DEFAULT 0,
  accepted_submissions INT NOT NULL DEFAULT 0,
  PRIMARY KEY (city_id, stat_date),
  KEY idx_cds_date (stat_date),
  CONSTRAINT fk_cds_city FOREIGN KEY (city_id) REFERENCES city(city_id) ON DELETE CASCADE,
  CONSTRAINT chk_cds_counts CHECK (accepted_submissions <= total_submissions)
) ENGINE=InnoDB;

CREATE TABLE city_follow (
  user_id     INT NOT NULL,
  city_id     INT NOT NULL,
  followed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, city_id),
  CONSTRAINT fk_cf_user FOREIGN KEY (user_id) REFERENCES app_user(user_id) ON DELETE CASCADE,
  CONSTRAINT fk_cf_city FOREIGN KEY (city_id) REFERENCES city(city_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------------------------------------- triggers
DELIMITER $$

-- integrity: submission.platform_id must match its account's platform
CREATE TRIGGER trg_submission_check BEFORE INSERT ON submission
FOR EACH ROW
BEGIN
  DECLARE acct_platform INT;
  SELECT platform_id INTO acct_platform FROM platform_account WHERE account_id = NEW.account_id;
  IF acct_platform <> NEW.platform_id THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'submission platform does not match account platform';
  END IF;
END$$

-- keep the daily rollup current as submissions arrive
-- (only sees the account's city at insert time: run sp_rebuild_city_stats() after locations are resolved)
CREATE TRIGGER trg_submission_rollup AFTER INSERT ON submission
FOR EACH ROW
BEGIN
  DECLARE acct_city INT;
  DECLARE acc BOOLEAN;
  SELECT city_id INTO acct_city FROM platform_account WHERE account_id = NEW.account_id;
  SELECT is_accepted INTO acc FROM verdict WHERE verdict_id = NEW.verdict_id;
  IF acct_city IS NOT NULL THEN
    INSERT INTO city_daily_stats (city_id, stat_date, total_submissions, accepted_submissions)
    VALUES (acct_city, DATE(NEW.submitted_at), 1, IF(acc, 1, 0))
    ON DUPLICATE KEY UPDATE
      total_submissions    = total_submissions + 1,
      accepted_submissions = accepted_submissions + IF(acc, 1, 0);
  END IF;
END$$

-- ---------------------------------------------------------------------------------------------------- procedures
 -- recompute the whole rollup from submissions, all-or-nothing.
-- call it after a batch of city resolutions (new city_alias rows, admin merges, re-resolve job)
CREATE PROCEDURE sp_rebuild_city_stats()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
  DELETE FROM city_daily_stats;
  INSERT INTO city_daily_stats (city_id, stat_date, total_submissions, accepted_submissions)
```

```
  SELECT a.city_id, DATE(s.submitted_at), COUNT(*), SUM(v.is_accepted)
  FROM submission s
  JOIN platform_account a ON a.account_id = s.account_id
  JOIN verdict v          ON v.verdict_id  = s.verdict_id
  WHERE a.city_id IS NOT NULL
  GROUP BY a.city_id, DATE(s.submitted_at);
  COMMIT;
END$$

DELIMITER ;

-- ---------------------------------------------------------------------------------------------------- views
-- city panel headline stats (submissions, users, accepted %)
CREATE VIEW v_city_summary AS
SELECT c.city_id, c.name AS city, co.name AS country, c.latitude, c.longitude,
       COUNT(s.submission_id)                          AS submissions,
       COUNT(DISTINCT s.account_id)                    AS users,
       ROUND(100 * SUM(v.is_accepted) / COUNT(*), 0)   AS accepted_pct
FROM city c
JOIN country co          ON co.country_id = c.country_id
JOIN platform_account a  ON a.city_id = c.city_id
JOIN submission s        ON s.account_id = a.account_id
JOIN verdict v           ON v.verdict_id = s.verdict_id
GROUP BY c.city_id, c.name, co.name, c.latitude, c.longitude;

-- "top topics" list in the city panel (a submission counts once per tag of its problem)
CREATE VIEW v_city_tag_counts AS
SELECT a.city_id, t.name AS tag, COUNT(*) AS submissions
FROM platform_account a
JOIN submission s   ON s.account_id = a.account_id
JOIN problem_tag pt ON pt.problem_id = s.problem_id
JOIN tag t          ON t.tag_id = pt.tag_id
WHERE a.city_id IS NOT NULL
GROUP BY a.city_id, t.name;
```

db/migrations/002_live_sync.sql — TESTING lifecycle, outbox, job log, geocode cache

```
-- GCode: live synchronization additions (run after schema.sql)
--
--   npm run db:migrate
--
-- Safe to run more than once: every statement is IF NOT EXISTS / OR REPLACE / IGNORE.
--
-- What it adds
--   1. TESTING verdict, so in-queue submissions are stored and later UPDATEd to their final verdict
--   2. trg_submission_verdict_change: keeps city_daily_stats.accepted_submissions right when a verdict changes
--   3. ingest_outbox: raw Codeforces JSON waiting to be copied into MongoDB (outbox pattern)
--   4. sync_run: one row per background job run, read by the admin dashboard
--   5. geocode_cache: city/country text we already asked the geocoder about (hit or miss)
--   6. sp_resolve_account_locations: links UNRESOLVED accounts to cities through city_alias
--   7. v_city_summary / v_city_tag_counts now skip TESTING, matching the MongoDB analytics

-- ---------------------------------------------------------------------------------------------------- 1. TESTING verdict
INSERT IGNORE INTO verdict (code, is_accepted) VALUES ('TESTING', FALSE);

-- ---------------------------------------------------------------------------------------------------- 3. outbox (MySQL -> MongoDB)
-- The poller writes the raw API JSON here in the same transaction as the relational rows,
-- so MySQL and the outbox can never disagree. outbox-to-mongo.js copies pending rows into
-- MongoDB (upsert by id) and stamps processed_at.
CREATE TABLE IF NOT EXISTS ingest_outbox (
  outbox_id    BIGINT AUTO_INCREMENT PRIMARY KEY,
  kind         ENUM('submission','user','problem') NOT NULL,
  external_key VARCHAR(64) NOT NULL,               -- submission id, handle or problem key
  payload      JSON NOT NULL,                      -- exactly what Codeforces returned
  created_at   DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  processed_at DATETIME(3) NULL,                   -- NULL = not yet in MongoDB
  KEY idx_outbox_pending (processed_at, outbox_id)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------------------------------------- 4. job log for the dashboard
CREATE TABLE IF NOT EXISTS sync_run (
  run_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  job         ENUM('poll','mongo','analytics','geocode','cleanup') NOT NULL,
  started_at  DATETIME(3) NOT NULL,
```

```
  finished_at DATETIME(3) NOT NULL,
  status      ENUM('OK','FAILED') NOT NULL,
  fetched     INT NOT NULL DEFAULT 0,
  inserted    INT NOT NULL DEFAULT 0,
  updated     INT NOT NULL DEFAULT 0,
  message     VARCHAR(500) NULL,
  KEY idx_run_job_time (job, started_at),
  KEY idx_run_time (started_at),
  CONSTRAINT chk_run_counts CHECK (fetched >= 0 AND inserted >= 0 AND updated >= 0),
  CONSTRAINT chk_run_time   CHECK (finished_at >= started_at)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------------------------------------- 5. geocoder cache
-- Nominatim allows one request per second, so never ask about the same text twice.
CREATE TABLE IF NOT EXISTS geocode_cache (
  raw_city     VARCHAR(150) NOT NULL,
  raw_country  VARCHAR(100) NOT NULL,
  city_id      INT NULL,                            -- NULL = geocoder found nothing
  attempted_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (raw_city, raw_country),
  CONSTRAINT fk_geocache_city FOREIGN KEY (city_id) REFERENCES city(city_id) ON DELETE SET NULL
) ENGINE=InnoDB;

DELIMITER $$

-- ---------------------------------------------------------------------------------------------------- 2. verdict lifecycle trigger
-- TESTING -> OK adds 1 to accepted_submissions, OK -> WRONG_ANSWER removes 1,
-- and total_submissions never changes on UPDATE (the row was already counted on INSERT).
DROP TRIGGER IF EXISTS trg_submission_verdict_change$$
CREATE TRIGGER trg_submission_verdict_change AFTER UPDATE ON submission
FOR EACH ROW
BEGIN
  DECLARE acct_city INT;
  DECLARE old_acc BOOLEAN;
  DECLARE new_acc BOOLEAN;

  IF OLD.verdict_id <> NEW.verdict_id THEN
    SELECT city_id INTO acct_city FROM platform_account WHERE account_id = NEW.account_id;
    SELECT is_accepted INTO old_acc FROM verdict WHERE verdict_id = OLD.verdict_id;
    SELECT is_accepted INTO new_acc FROM verdict WHERE verdict_id = NEW.verdict_id;

    IF acct_city IS NOT NULL AND old_acc <> new_acc THEN
      UPDATE city_daily_stats
         SET accepted_submissions = accepted_submissions + IF(new_acc, 1, -1)
       WHERE city_id = acct_city AND stat_date = DATE(NEW.submitted_at);
    END IF;
  END IF;
END$$

-- ---------------------------------------------------------------------------------------------------- 6. location resolver
-- After the geocoder adds new city_alias rows, attach every UNRESOLVED account whose
-- raw text now matches. Returns the number of accounts it resolved.
DROP PROCEDURE IF EXISTS sp_resolve_account_locations$$
CREATE PROCEDURE sp_resolve_account_locations(OUT resolved INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
  UPDATE platform_account a
  JOIN country co    ON co.name = a.raw_country
  JOIN city_alias al ON al.raw_text = a.raw_city AND al.country_id = co.country_id
  JOIN city c        ON c.city_id = al.city_id AND c.country_id = co.country_id
     SET a.city_id  =  c.city_id,
         a.country_id = c.country_id,
         a.location_status = 'CITY_RESOLVED'
   WHERE a.location_status = 'UNRESOLVED';
  SET resolved = ROW_COUNT();
  COMMIT;
END$$

DELIMITER ;
```

```
-- ---------------------------------------------------------------------------------------------- 7. views without TESTING
CREATE OR REPLACE VIEW v_city_summary AS
SELECT c.city_id, c.name AS city, co.name AS country, c.latitude, c.longitude,
       COUNT(s.submission_id)                          AS submissions,
       COUNT(DISTINCT s.account_id)                    AS users,
       ROUND(100 * SUM(v.is_accepted) / COUNT(*), 0)   AS accepted_pct
FROM city c
JOIN country co        ON co.country_id = c.country_id
JOIN platform_account a ON a.city_id = c.city_id
JOIN submission s      ON s.account_id = a.account_id
JOIN verdict v          ON v.verdict_id = s.verdict_id
WHERE v.code <> 'TESTING'
GROUP BY c.city_id, c.name, co.name, c.latitude, c.longitude;

CREATE OR REPLACE VIEW v_city_tag_counts AS
SELECT a.city_id, t.name AS tag, COUNT(*) AS submissions
FROM platform_account a
JOIN submission s   ON s.account_id = a.account_id
JOIN verdict v      ON v.verdict_id = s.verdict_id
JOIN problem_tag pt ON pt.problem_id = s.problem_id
JOIN tag t          ON t.tag_id = pt.tag_id
WHERE a.city_id IS NOT NULL AND v.code <> 'TESTING'
GROUP BY a.city_id, t.name;
```

db/migrations/003_roles.sql — role-based access control inside MySQL

```
-- GCode: role-based access control inside MySQL (run after schema.sql and 002_live_sync.sql)
--
--   mysql -u root -p gcode < db/migrations/003_roles.sql
--
-- Two roles, enforced by the database itself (not only by the app):
--   gcode_viewer  SELECT only, on the tables and views the public pages read
--   gcode_admin   SELECT everywhere + write access only to the sync tables the dashboard uses
--
-- The Next.js app connects as gcode_viewer for public routes (/api/problems, /api/tags) and as
-- gcode_admin for /api/admin/*. So even if a bug let a viewer reach a write query, MySQL would
-- refuse it. The worker and the login route keep using the main DB_USER account.
--
-- BEFORE RUNNING: change the two passwords below, then put the same values in .env.local
-- (DB_VIEWER_USER / DB_VIEWER_PASSWORD and DB_ADMIN_USER / DB_ADMIN_PASSWORD).
-- Safe to run more than once.

----
CREATE ROLE IF NOT EXISTS 'gcode_viewer', 'gcode_admin';

-- viewer: read-only, and no access to app_user (password hashes) or the sync tables
GRANT SELECT ON gcode.problem          TO 'gcode_viewer';
GRANT SELECT ON gcode.problem_tag      TO 'gcode_viewer';
GRANT SELECT ON gcode.tag              TO 'gcode_viewer';
GRANT SELECT ON gcode.submission       TO 'gcode_viewer';
GRANT SELECT ON gcode.verdict          TO 'gcode_viewer';
GRANT SELECT ON gcode.platform         TO 'gcode_viewer';
GRANT SELECT ON gcode.country          TO 'gcode_viewer';
GRANT SELECT ON gcode.city             TO 'gcode_viewer';
GRANT SELECT ON gcode.city_daily_stats TO 'gcode_viewer';
GRANT SELECT ON gcode.v_city_summary    TO 'gcode_viewer';
GRANT SELECT ON gcode.v_city_tag_counts TO 'gcode_viewer';

-- admin: read everything, write only what the dashboard's sync and job log need
GRANT SELECT ON gcode.*                TO 'gcode_admin';
GRANT INSERT ON gcode.sync_run         TO 'gcode_admin';
GRANT INSERT, UPDATE ON gcode.ingest_outbox TO 'gcode_admin';

-- ---------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-- (CHANGE THESE PASSWORDS)
CREATE USER IF NOT EXISTS 'gcode_viewer_user'@'localhost' IDENTIFIED BY 'change-me-viewer';
CREATE USER IF NOT EXISTS 'gcode_admin_user'@'localhost'  IDENTIFIED BY 'change-me-admin';

GRANT 'gcode_viewer' TO 'gcode_viewer_user'@'localhost';
GRANT 'gcode_admin'  TO 'gcode_admin_user'@'localhost';

-- a role only applies to a connection if it is a default role of the account
SET DEFAULT ROLE 'gcode_viewer' TO 'gcode_viewer_user'@'localhost';
```

```
SET DEFAULT ROLE 'gcode_admin'  TO 'gcode_admin_user'@'localhost';

-- ---------------------------------------------------------------------------------------------------- demo / viva checks
-- Run these while connected as each account (mysql -u gcode_viewer_user -p gcode):
--
--   as gcode_viewer_user:
--     SELECT COUNT(*) FROM problem;                          -- works
--     SELECT * FROM app_user;                                -- ERROR 1142: SELECT command denied
--     INSERT INTO tag (name) VALUES ('hacked');              -- ERROR 1142: INSERT command denied
--     DELETE FROM submission;                                -- ERROR 1142: DELETE command denied
--
--   as gcode_admin_user:
--     SELECT COUNT(*) FROM sync_run;                         -- works
--     INSERT INTO sync_run (job, started_at, finished_at, status)
--       VALUES ('cleanup', NOW(), NOW(), 'OK');              -- works
--     DELETE FROM submission;                                -- ERROR 1142: DELETE command denied
--
--   SHOW GRANTS FOR 'gcode_viewer_user'@'localhost' USING 'gcode_viewer';
--   SHOW GRANTS FOR 'gcode_admin_user'@'localhost'  USING 'gcode_admin';
```

SQL DML Statements

Every statement below was executed on the project database during the test run; each figure shows the statement and its real output. INSERT/UPDATE/DELETE demonstrations ran inside a transaction that was rolled back afterwards. The commands cover DML (INSERT, UPDATE, DELETE, SELECT), TCL (START TRANSACTION, SAVEPOINT, COMMIT, ROLLBACK) and DCL (GRANT, REVOKE).

INSERT, UPDATE and triggers (TESTING → final verdict)

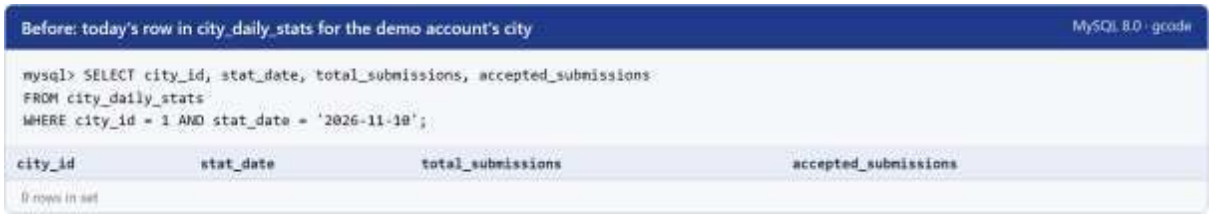


Figure 3: Before: today's row in city_daily_stats for the demo account's city

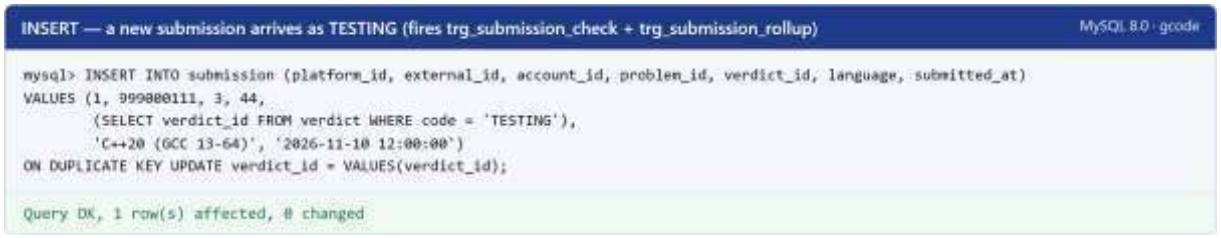


Figure 4: INSERT — a new submission arrives as TESTING (fires trg_submission_check + trg_submission_rollup)

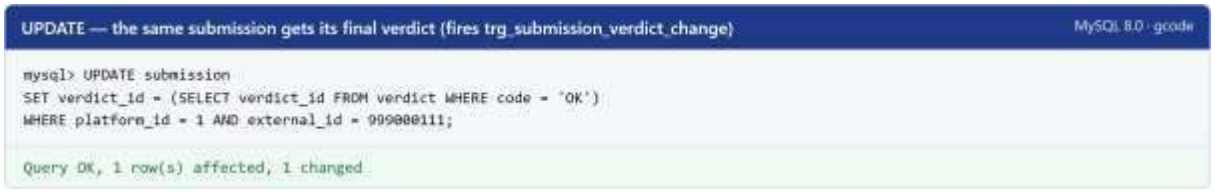


Figure 5: UPDATE — the same submission gets its final verdict (fires trg_submission_verdict_change)

After: total counted once, accepted moved by the trigger — MySQL 8.0 · gcode

```
mysql> SELECT city_id, stat_date, total_submissions, accepted_submissions
FROM city_daily_stats
WHERE city_id = 1 AND stat_date = '2026-11-10';
```

city_id	stat_date	total_submissions	accepted_submissions
1	2026-11-10	1	1

1 row in set

Figure 6: After: total counted once, accepted moved by the trigger

Duplicate protection — same Codeforces id again is an UPDATE, not a second row — MySQL 8.0 · gcode

```
mysql> SELECT COUNT(*) AS rows_for_id_999000111
FROM submission WHERE platform_id = 1 AND external_id = 999000111;
```

rows_for_id_999000111
1

1 row in set

Figure 7: Duplicate protection — same Codeforces id again is an UPDATE, not a second row

DELETE

DELETE — hourly cleanup of processed outbox rows (worker job) — MySQL 8.0 · gcode

```
mysql> DELETE FROM ingest_outbox
WHERE processed_at IS NOT NULL
  AND processed_at < UTC_TIMESTAMP() - INTERVAL 1 MINUTE;
```

```
Query OK, 625 row(s) affected, 0 changed
```

Figure 8: DELETE — hourly cleanup of processed outbox rows (worker job)

Join queries (INNER / LEFT JOIN, GROUP BY, HAVING, ORDER BY, aggregates, view)

JOIN + GROUP BY + ORDER BY — busiest cities (view v_city_summary) — MySQL 8.0 · gcode

```
mysql> SELECT city, country, submissions, users, accepted_pct
FROM v_city_summary
ORDER BY submissions DESC, city
LIMIT 10;
```

city	country	submissions	users	accepted_pct
Pune	India	17	5	24
Delhi	India	14	8	43
Ho Chi Minh City	Vietnam	13	6	38
Cairo	Egypt	12	9	67
Hong Kong	Hong Kong	10	2	10
Dhaka	Bangladesh	9	7	33
Hyderabad	India	9	4	67
Moscow	Russia	9	5	33
Almaty	Kazakhstan	8	4	38
Chennai	India	7	3	71

10 rows in set

Figure 9: JOIN + GROUP BY + ORDER BY — busiest cities (view v_city_summary)

LEFT JOIN — where accounts come from, including those with no usable city — MySQL 8.0 · gcode

```
mysql> SELECT a.location_status, COUNT(*) AS accounts,
       COUNT(c.city_id) AS mapped_to_city
FROM platform_account a
LEFT JOIN city c ON c.city_id = a.city_id
GROUP BY a.location_status
ORDER BY accounts DESC;
```

location_status	accounts	mapped_to_city
UNKNOWN	607	0
CITY_RESOLVED	223	223
COUNTRY_ONLY	76	0
UNRESOLVED	50	0

4 rows in set

Figure 10: LEFT JOIN — where accounts come from, including those with no usable city

JOIN + GROUP BY + HAVING — countries with at least 15 Codeforces accounts — MySQL 8.0 · gcode

```
mysql> SELECT co.name AS country,
       COUNT(*) AS accounts,
       ROUND(AVG(a.rating)) AS avg_rating,
       SUM(a.location_status = 'CITY_RESOLVED') AS with_city
FROM platform_account a
JOIN country co ON co.country_id = a.country_id
GROUP BY co.country_id, co.name
HAVING COUNT(*) >= 15
ORDER BY accounts DESC;
```

country	accounts	avg_rating	with_city
India	111	1239	74
China	37	1558	18
Egypt	33	1311	23
Russia	23	1647	17
Bangladesh	20	1192	10

5 rows in set

Figure 11: JOIN + GROUP BY + HAVING — countries with at least 15 Codeforces accounts

Nested and correlated subqueries

Nested subquery — problems harder than the average problem anyone solved MySQL 8.0 · gcode

```
mysql> SELECT p.external_key, p.name, p.rating
FROM problem p
WHERE p.rating > (SELECT AVG(p2.rating)
                  FROM submission s
                  JOIN verdict v  ON v.verdict_id = s.verdict_id AND v.is_accepted
                  JOIN problem p2 ON p2.problem_id = s.problem_id)
  AND p.problem_id IN (SELECT problem_id FROM submission)
ORDER BY p.rating DESC
LIMIT 10;
```

external_key	name	rating
1975I	Mind Bloom	3500
1385G	Kuroni and Antihype	3500
1500F	Cupboards Jumps	3500
2239E	The end of this world,	3300
1656H	Equal LCM Subsets	3200
2096F	Wonderful Impostors	3100
1770F	Koxia and Sequence	3100
896E	Welcome home, Chtholly	3100
2077D	Maximum Polygon	3100
1975G	Zimpha Fan Club	3000

10 rows in set

Figure 12: Nested subquery — problems harder than the average problem anyone solved

Correlated subquery — accounts rated above their own country's average MySQL 8.0 · gcode

```
mysql> SELECT a.handle, co.name AS country, a.rating,
       (SELECT ROUND(AVG(a2.rating)) FROM platform_account a2
         WHERE a2.country_id = a.country_id) AS country_avg
FROM platform_account a
JOIN country co ON co.country_id = a.country_id
WHERE a.rating > (SELECT AVG(a2.rating) FROM platform_account a2
                  WHERE a2.country_id = a.country_id)
ORDER BY a.rating DESC
LIMIT 10;
```

handle	country	rating	country_avg
op__op	China	2537	1558
eriksuenderhauf	Germany	2364	1874
Mathematical_analogy	India	2341	1239
Aldk	Kazakhstan	2325	1361
ender_shayan	Iran	2324	1514
Tong_Yuxin	China	2310	1558
Baraa-Ahmed	Egypt	2303	1311
Krzychuo	Poland	2256	1557
yaufung	Hong Kong	2252	1585
surgutti	Poland	2247	1557

10 rows in set

Figure 13: Correlated subquery — accounts rated above their own country's average

Common table expressions (CTE and recursive CTE)

CTE — most solved problems (same query as /api/problems) — MySQL 8.0 · gcode

```
mysql> WITH solved AS (
  SELECT s.problem_id, COUNT(DISTINCT s.account_id) AS solvers
  FROM submission s
  JOIN verdict v ON v.verdict_id = s.verdict_id AND v.is_accepted
  GROUP BY s.problem_id
)
SELECT p.external_key, p.name, p.rating, sv.solvers,
       GROUP_CONCAT(t.name ORDER BY t.name SEPARATOR ', ') AS tags
FROM solved sv
JOIN problem p          ON p.problem_id = sv.problem_id
LEFT JOIN problem_tag pt ON pt.problem_id = p.problem_id
LEFT JOIN tag t          ON t.tag_id = pt.tag_id
GROUP BY p.problem_id, p.external_key, p.name, p.rating, sv.solvers
ORDER BY sv.solvers DESC, p.rating
LIMIT 8;
```

external_key	name	rating	solvers	tags
2280A	Are They the Same?	NULL	15	implementation
2280C	Reverse and Double	NULL	13	binary search
2284A1	1SK4JR3B (Easy Version)	NULL	11	greedy, math, number theory
2280B	Secret String Patterns	NULL	10	interactive
231A	Team	800	8	brute force, greedy
2260A	Monocarp's Contest	800	8	implementation
50A	Domino piling	800	7	greedy, math
4A	Watermelon	800	6	brute force, math

8 rows in set

Figure 14: CTE — most solved problems (same query as /api/problems)

Recursive CTE — rating ladder: how many problems exist at each 100-point step — MySQL 8.0 · gcode

```
mysql> WITH RECURSIVE ladder (rating) AS (
  SELECT 800
  UNION ALL
  SELECT rating + 100 FROM ladder WHERE rating < 3500
)
SELECT l.rating, COUNT(p.problem_id) AS problems
FROM ladder l
LEFT JOIN problem p ON p.rating = l.rating
GROUP BY l.rating
ORDER BY l.rating
LIMIT 12;
```

rating	problems
800	1110
900	354
1000	412
1100	449
1200	463
1300	476
1400	468
1500	478
1600	531
1700	520
1800	497
1900	539

12 rows in set

Figure 15: Recursive CTE — rating ladder: how many problems exist at each 100-point step

Full-text search

FULLTEXT search — the Problems page search box — MySQL 8.0 · gcode

```
mysql> SELECT external_key, name, rating
FROM problem
WHERE MATCH(name) AGAINST ('+array* +permut*' IN BOOLEAN MODE)
LIMIT 8;
```

external_key	name	rating
21978	Array and Permutation	1100
1867A	green_gold_dog, array and permutation	800

2 rows in set

Figure 16: FULLTEXT search — the Problems page search box

Stored procedure, triggers and routines

Stored procedure — sp_resolve_account_locations(OUT resolved) — MySQL 8.0 · gcode

```
mysql> CALL sp_resolve_account_locations(@resolved);
Query OK, 0 row(s) affected, 0 changed
```

Figure 17: Stored procedure — sp_resolve_account_locations(OUT resolved)

Reading the OUT parameter — MySQL 8.0 · gcode

```
mysql> SELECT @resolved AS accounts_resolved;
```

accounts_resolved
0

1 row in set

Figure 18: Reading the OUT parameter

Triggers on submission — MySQL 8.0 · gcode

```
mysql> SELECT TRIGGER_NAME, ACTION_TIMING, EVENT_MANIPULATION, EVENT_OBJECT_TABLE
FROM information_schema.TRIGGERS
WHERE TRIGGER_SCHEMA = 'gcode';
```

TRIGGER_NAME	ACTION_TIMING	EVENT_MANIPULATION	EVENT_OBJECT_TABLE
trg_submission_check	BEFORE	INSERT	submission
trg_submission_rollup	AFTER	INSERT	submission
trg_submission_verdict_change	AFTER	UPDATE	submission

3 rows in set

Figure 19: Triggers on submission

Stored procedures — MySQL 8.0 · gcode

```
mysql> SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gcode';
```

ROUTINE_NAME	ROUTINE_TYPE
sp_rebuild_city_stats	PROCEDURE
sp_resolve_account_locations	PROCEDURE

2 rows in set

Figure 20: Stored procedures

Constraints enforced by the database

CHECK constraint — the database rejects an invalid latitude — MySQL 8.0 · gcode

```
mysql> INSERT INTO city (country_id, name, latitude, longitude)
VALUES (1, 'Nowhere', 123.0, 10.0);
```

```
ERROR 3819 (HY000): Check constraint 'chk_city_lat' is violated.
```

Figure 21: CHECK constraint — the database rejects an invalid latitude

FOREIGN KEY — a submission for an account that does not exist is rejected — MySQL 8.0 · gcode

```
mysql> INSERT INTO submission (platform_id, external_id, account_id, problem_id, verdict_id, submitted_at)
VALUES (1, 1, 999999, 1, 1, NOW());
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`gcode`.`submission`, CONSTRAINT
`fk_sub_account` FOREIGN KEY (`account_id`) REFERENCES `platform_account` (`account_id`))
```

Figure 22: FOREIGN KEY — a submission for an account that does not exist is rejected

Role-based access control in MySQL

As gcode_viewer_user: reading problems is allowed — MySQL 8.0 · gcode

```
mysql> SELECT COUNT(*) AS problems FROM problem;
```

problems
11590

1 row in set

Figure 23: As gcode_viewer_user: reading problems is allowed

As gcode_viewer_user: reading password hashes is denied — MySQL 8.0 · gcode

```
mysql> SELECT email, password_hash FROM app_user;
```

```
ERROR 1142 (42000): SELECT command denied to user 'gcode_viewer_user'@'localhost' for table 'app_user'
```

Figure 24: As gcode_viewer_user: reading password hashes is denied

As gcode_viewer_user: writing is denied — MySQL 8.0 · gcode

```
mysql> INSERT INTO tag (name) VALUES ('hacked');
```

```
ERROR 1142 (42000): INSERT command denied to user 'gcode_viewer_user'@'localhost' for table 'tag'
```

Figure 25: As gcode_viewer_user: writing is denied

Grants of the two database roles — MySQL 8.0 · gcode

```
mysql> SELECT GRANTEE, TABLE_NAME, PRIVILEGE_TYPE
FROM information_schema.TABLE_PRIVILEGES
WHERE GRANTEE LIKE '%gcode_admin%' AND TABLE_SCHEMA = 'gcode';
```

GRANTEE	TABLE_NAME	PRIVILEGE_TYPE
'gcode_admin'@'%'	sync_run	INSERT
'gcode_admin'@'%'	ingest_outbox	INSERT
'gcode_admin'@'%'	ingest_outbox	UPDATE

3 rows in set

Figure 26: Grants of the two database roles

Transactions (START TRANSACTION, SAVEPOINT, COMMIT, ROLLBACK)

Transaction: ROLLBACK undoes the DELETE (and the trigger updates it caused) — MySQL 8.0 · gcode

```
mysql> START TRANSACTION; root
Query OK, 0 row(s) affected
mysql> DELETE FROM submission WHERE account_id = 62;   -- all of 'SaltyDonut''s submissions root
Query OK, 11 row(s) affected
mysql> SELECT COUNT(*) AS submissions_of_SaltyDonut FROM submission WHERE account_id = 62; root
```

submissions_of_SaltyDonut
0

```
mysql> ROLLBACK; root
Query OK, 0 row(s) affected
mysql> SELECT COUNT(*) AS submissions_of_SaltyDonut FROM submission WHERE account_id = 62; root
```

submissions_of_SaltyDonut
11

Figure 27: Transaction: ROLLBACK undoes the DELETE (and the trigger updates it caused)

Transaction: SAVEPOINT, partial ROLLBACK, then COMMIT — MySQL 8.0 · gcode

```
mysql> START TRANSACTION; root
Query OK, 0 row(s) affected
mysql> INSERT INTO tag (name) VALUES ('demo-kept'); root
Query OK, 1 row(s) affected
mysql> SAVEPOINT before_second; root
Query OK, 0 row(s) affected
mysql> INSERT INTO tag (name) VALUES ('demo-undone'); root
Query OK, 1 row(s) affected
mysql> ROLLBACK TO SAVEPOINT before_second; root
Query OK, 0 row(s) affected
mysql> COMMIT; root
Query OK, 0 row(s) affected
mysql> SELECT tag_id, name FROM tag WHERE name LIKE 'demo-%'; root
```

tag_id	name
33081	demo-kept

Figure 28: Transaction: SAVEPOINT, partial ROLLBACK, then COMMIT

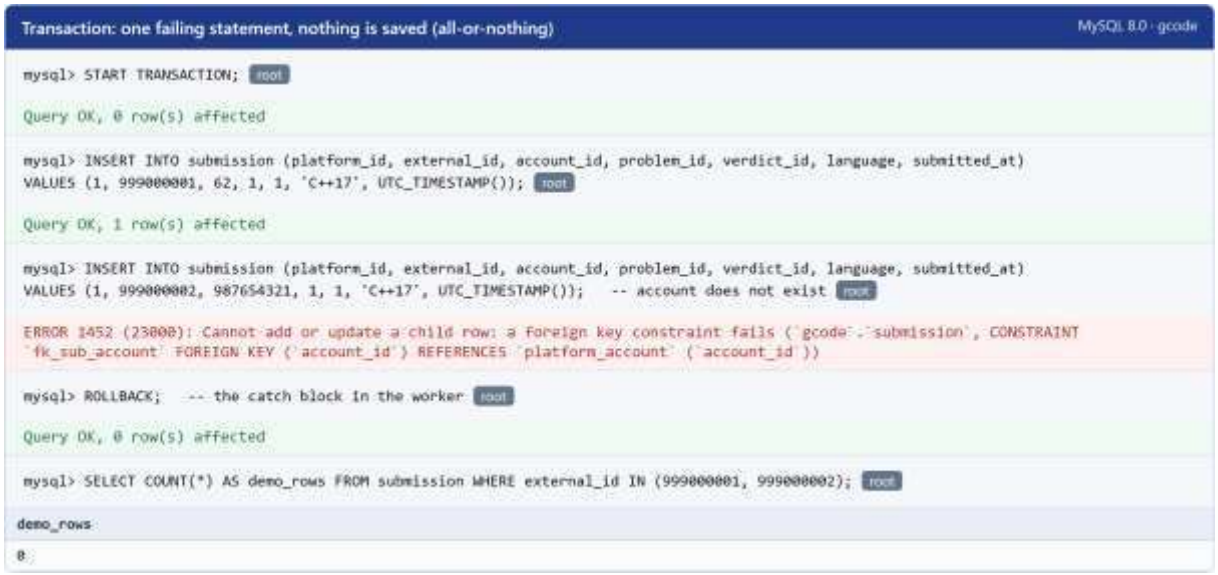


Figure 29: Transaction: one failing statement, nothing is saved (all-or-nothing)

Live poller: every poll is one MySQL transaction (commit, or roll back everything on error) — scripts/sync/poll-submissions.js

```
    // 4. one transaction for everything
    await db.beginTransaction();
    try {
      for (const rawUser of newUsers) {
        await upsertAccount(db, platformId, normalizeUser(rawUser), rawUser);
      }
      if (newUsers.length) {
        const [rows] = await db.query(
          "SELECT handle, account_id FROM platform_account WHERE platform_id = ? AND handle IN (?)",
          [platformId, newUsers.map((u) => u.handle)]
        );
        for (const r of rows) accountIds.set(r.handle.toLowerCase(), r.account_id);
      }

      for (const { raw: s, norm } of submissions) {
        const accountId = accountIds.get(norm.participants[0].toLowerCase());
        if (!accountId) continue;            // handle vanished from Codeforces

        let problemId = problemIds.get(norm.problemExternalKey);
        if (!problemId) {
          problemId = await upsertProblem(db, platformId, s.problem);
          problemIds.set(norm.problemExternalKey, problemId);
        }

        const verdict = s.verdict ?? "TESTING";
        const verdictId = await getVerdictId(db, state, verdict);

        // affectedRows: 1 = inserted, 2 = existing row changed, 0 = identical row
        const [result] = await db.execute(
          `INSERT INTO submission
             (platform_id, external_id, account_id, problem_id, verdict_id, language, submitted_at)
           VALUES (?, ?, ?, ?, ?, ?, ?)
           ON DUPLICATE KEY UPDATE verdict_id = VALUES(verdict_id)`,
          [platformId, s.id, accountId, problemId, verdictId, norm.language?.slice(0, 50) ?? null,
           toMySQLDate(norm.submittedAt)]
        );

        if (result.affectedRows === 1) inserted++;
        if (result.affectedRows === 2) updated++;
        if (result.affectedRows > 0) await outbox(db, "submission", s.id, s);
      }

      await db.commit();
    } catch (error) {
      await db.rollback();
      throw error;
    }
```

Figure 30: Live poller: each poll is one MySQL transaction — COMMIT, or ROLLBACK everything on error (scripts/sync/poll-submissions.js)

Data Control Language: GRANT and REVOKE

GRANT and REVOKE on the gcode_viewer role (effect seen by gcode_viewer_user) — MySQL 8.0 · gcode

```
mysql> SHOW GRANTS FOR 'gcode_viewer'; root
Grants for gcode_viewer@%
GRANT USAGE ON *.* TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`city_daily_stats` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`city` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`country` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`platform` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`problem_tag` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`problem` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`submission` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`tag` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`v_city_summary` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`v_city_tag_counts` TO `gcode_viewer`@`%`
GRANT SELECT ON `gcode`.`verdict` TO `gcode_viewer`@`%`
mysql> SELECT COUNT(*) AS contests FROM contest; gcode_viewer_user
ERROR 1142 (42000): SELECT command denied to user 'gcode_viewer_user'@'localhost' for table 'contest'
mysql> GRANT SELECT ON gcode.contest TO 'gcode_viewer'; root
Query OK, 0 row(s) affected
mysql> SELECT COUNT(*) AS contests FROM contest; gcode_viewer_user
contests
2168
mysql> REVOKE SELECT ON gcode.contest FROM 'gcode_viewer'; root
Query OK, 0 row(s) affected
mysql> SELECT COUNT(*) AS contests FROM contest; gcode_viewer_user
ERROR 1142 (42000): SELECT command denied to user 'gcode_viewer_user'@'localhost' for table 'contest'
```

Figure 31: GRANT and REVOKE on the gcode_viewer role (effect seen by gcode_viewer_user)

Revoking the whole role from a user, then granting it back — MySQL 8.0 · gcode

```
mysql> REVOKE 'gcode_viewer' FROM 'gcode_viewer_user'@'localhost'; root
Query OK, 0 row(s) affected
mysql> -- log in: mysql -u gcode_viewer_user -p gcode gcode_viewer_user
ERROR 1044 (42000): Access denied for user 'gcode_viewer_user'@'localhost' to database 'gcode'
mysql> GRANT 'gcode_viewer' TO 'gcode_viewer_user'@'localhost'; root
Query OK, 0 row(s) affected
mysql> SET DEFAULT ROLE 'gcode_viewer' TO 'gcode_viewer_user'@'localhost'; root
Query OK, 0 row(s) affected
mysql> SELECT COUNT(*) AS problems FROM problem; gcode_viewer_user
problems
11569
```

Figure 32: Revoking the whole role from a user, then granting it back

DML used by the live sync worker (scripts/sync/poll-submissions.js)

```
-- one transaction per poll; duplicates become updates
INSERT INTO platform_account (platform_id, handle, raw_city, raw_country, city_id, country_id,
```

```
                              location_status, rating, last_synced_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, UTC_TIMESTAMP())
ON DUPLICATE KEY UPDATE raw_city = VALUES(raw_city), raw_country = VALUES(raw_country),
  city_id = VALUES(city_id), country_id = VALUES(country_id),
  location_status = VALUES(location_status), rating = VALUES(rating), last_synced_at = VALUES(last_synced_at);

INSERT INTO submission (platform_id, external_id, account_id, problem_id, verdict_id, language, submitted_at)
VALUES (?, ?, ?, ?, ?, ?, ?)
ON DUPLICATE KEY UPDATE verdict_id = VALUES(verdict_id);    -- affectedRows 1 = new, 2 = verdict changed

INSERT INTO ingest_outbox (kind, external_key, payload) VALUES (?, ?, ?);          -- raw JSON for MongoDB
UPDATE ingest_outbox SET processed_at = UTC_TIMESTAMP(3) WHERE outbox_id IN (?);  -- after the MongoDB upsert
DELETE FROM ingest_outbox WHERE processed_at < UTC_TIMESTAMP() - INTERVAL 1 DAY;  -- hourly cleanup
DELETE FROM sync_run WHERE started_at < UTC_TIMESTAMP() - INTERVAL 2 DAY;
CALL sp_resolve_account_locations(@resolved);  CALL sp_rebuild_city_stats();
```

NoSQL/ Vector Database Commands

MongoDB database `gcode`. Collections: `raw_submissions`, `raw_users`, `raw_problems`, `raw_contests` (raw API documents), `account_city` and `city_dim` (helper copies from MySQL), `city_analytics` (one precomputed document per city, read by the globe) and `saved_views` (user-owned documents). Each figure shows the command and its real output.

Collections, unique indexes and $jsonSchema validation



Figure 33: Collections in the gcode database



Figure 34: Unique indexes stop duplicate raw documents



Figure 35: city_analytics is created with a $jsonSchema validator (excerpt)

Aggregation pipeline that builds city_analytics

`scripts/build-city-analytics.cjs` first flattens `raw_submissions` into `tmp_city_submissions` ($match, $addFields, $lookup into account_city and raw_problems, $project, $out), then runs one pipeline per panel section — totals, topics, verdicts, mostSolved, languages, difficulty — each ending in $merge into a build collection, which is finally renamed over `city_analytics` in one step:

```
db.raw_submissions.aggregate([
  { $match: { verdict: { $type: "string", $ne: "TESTING" } } },
  { $addFields: { handleLower: { $toLower: { $arrayElemAt: ["$author.members.handle", 0] } },
                  key: { $concat: [{ $toString: "$problem.contestId" }, "$problem.index"] } } },
  { $lookup: { from: "account_city", localField: "handleLower", foreignField: "handleLower", as: "ac" } },
  { $match: { "ac.0": { $exists: true } } },
  { $lookup: { from: "raw_problems", localField: "key", foreignField: "key", as: "pd" } },
  { $project: { cityId: { $arrayElemAt: ["$ac.cityId", 0] }, verdict: 1, tags: "$problem.tags", ... } },
  { $out: "tmp_city_submissions" }
])

db.tmp_city_submissions.aggregate([             // "top topics" section
  { $unwind: "$tags" },
  { $group: { _id: { cityId: "$cityId", tag: "$tags" }, count: { $sum: 1 } } },
  { $sort: { count: -1, "_id.tag": 1 } },
  { $group: { _id: "$_id.cityId", topics: { $push: { tag: "$_id.tag", count: "$count" } } } },
  { $project: { _id: 0, cityId: "$_id", topics: { $slice: ["$topics", 6] } } },
  { $merge: { into: "city_analytics_build", on: "cityId", whenMatched: "merge", whenNotMatched: "discard" } }
])
```

```
db.city_analytics_build.renameCollection("city_analytics", true)    // atomic swap, validator + indexes kept
```

READ

READ — the document the side panel loads when a city is clicked — MongoDB 8.0 · gcode

```
gcode> db.city_analytics.findOne(
  { cityId: 16 },
  { _id: 0, city: 1, country: 1, totals: 1, topics: { $slice: 3 }, verdicts: { $slice: 3 }, languages: 1, updatedAt: 1 }
)
```


```
{
  "city": "Pune",
  "country": "India",
  "totals": {
    "submissions": 16,
    "users": 4,
    "accepted": 3,
    "acceptedPct": 19
  },
  "updatedAt": "2026-11-10T13:07:31.512Z",
  "topics": [
    {
      "tag": "brute force",
      "count": 11
    },
    {
      "tag": "graphs",
      "count": 11
    },
    {
      "tag": "combinatorics",
      "count": 2
    }
  ],
  "verdicts": [
    {
      "code": "WRONG_ANSWER",
      "count": 7
    },
    {
      "code": "OK",
      "count": 3
    },
    {
      "code": "MEMORY_LIMIT_EXCEEDED",
      "count": 2
    }
  ],
  "languages": [
    {
      "name": "C++",
      "count": 16
    }
  ]
}
```


Figure 36: READ — the document the side panel loads when a city is clicked

Aggregation — most practised topics across all cities — MongoDB 8.0 · gcode

```
gcode> db.city_analytics.aggregate([
  { $unwind: "$topics" },
  { $group: { _id: "$topics.tag", submissions: { $sum: "$topics.count" }, cities: { $sum: 1 } } },
  { $sort: { submissions: -1 } },
  { $limit: 6 }
])
```

```
[
  {
    "_id": "greedy",
    "submissions": 94,
    "cities": 48
  },
  {
    "_id": "math",
    "submissions": 79,
    "cities": 43
  },
  {
    "_id": "brute force",
    "submissions": 73,
    "cities": 39
  },
  {
    "_id": "implementation",
    "submissions": 65,
    "cities": 36
  },
  {
    "_id": "dp",
    "submissions": 54,
    "cities": 31
  },
  {
    "_id": "binary search",
    "submissions": 49,
    "cities": 29
  }
]
```

Figure 37: Aggregation — most practised topics across all cities

UPDATE (upsert by Codeforces id — no duplicate documents)

UPDATE (upsert) — the outbox sync replaces a submission by its Codeforces id — MongoDB 8.0 · gcode

```
gcode> db.raw_submissions.updateOne(
  { id: 397846016 },
  { $set: { verdict: "TESTING", _syncedAt: new Date() } },
  { upsert: true }
)
```

```
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedCount": 0
}
```

Figure 38: UPDATE (upsert) — the outbox sync replaces a submission by its Codeforces id

Create, Read, Update, Delete on saved_views (with schema validation)

saved_views — CREATE a user's saved globe view (validated by $jsonSchema) MongoDB 8.0 · gcode

```
gcode> db.saved_views.insertOne({
  userId: 1, name: "Asia, rated 1600+",
  filters: { continent: "Asia", minRating: 1600 },
  camera: { lat: 22.5, lng: 88.3, zoom: 1.6 },
  createdAt: new Date()
})
```

```
{
  "acknowledged": true,
  "insertedId": "6af31723a89e2d8c0eac9951"
}
```

Figure 39: saved_views — CREATE a user's saved globe view (validated by $jsonSchema)

saved_views — READ the user's views MongoDB 8.0 · gcode

```
gcode> db.saved_views.find({ userId: 1 }, { _id: 0, name: 1, filters: 1, camera: 1 })
```

```
[
  {
    "name": "Asia, rated 1600+",
    "filters": {
      "continent": "Asia",
      "minRating": 1600
    },
    "camera": {
      "lat": 22.5,
      "lng": 88.3,
      "zoom": 1.6
    }
  }
]
```

Figure 40: saved_views — READ the user's views

saved_views — UPDATE the view's name and camera MongoDB 8.0 · gcode

```
gcode> db.saved_views.updateOne(
  { userId: 1, name: "Asia, rated 1600+" },
  { $set: { name: "South Asia, 1600+", "camera.zoom": 2.2, updatedAt: new Date() } }
)
```

```
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1
}
```

Figure 41: saved_views — UPDATE the view's name and camera

Validation — a document breaking the schema is rejected (lat must be -90..90) MongoDB 8.0 · gcode

```
gcode> db.saved_views.insertOne({ userId: 1, name: "Bad", filters: {}, camera: { lat: 500, lng: 0 }, createdAt: new Date() })
```

```
MongoServerError: Document failed validation
```

Figure 42: Validation — a document breaking the schema is rejected (lat must be -90..90)

saved_views — DELETE the view MongoDB 8.0 · gcode

```
gcode> db.saved_views.deleteOne({ userId: 1, name: "South Asia, 1600+" })
```

```
{
  "acknowledged": true,
  "deletedCount": 1
}
```

Figure 43: saved_views — DELETE the view

Document counts

Document counts per collection MongoDB 8.0 · gcode

```
gcode> ["raw_submissions","raw_users","raw_problems","raw_contests","city_analytics"]
  .map(c => ({ collection: c, documents: db[c].countDocuments() }))
```

```
[
  {
    "collection": "raw_submissions",
    "documents": 1388
  },
  {
    "collection": "raw_users",
    "documents": 945
  },
  {
    "collection": "raw_problems",
    "documents": 11532
  },
  {
    "collection": "raw_contests",
    "documents": 2168
  },
  {
    "collection": "city_analytics",
    "documents": 110
  }
]
```

Figure 44: Document counts per collection

Results

Resulting Tables' Screenshots

State of the database after the live worker had run on real Codeforces data: 1,708 submissions, 1,140 accounts, 11,603 problems, 2,196 contests, 153 cities in 47 countries, 38 tags, and 138 city documents in MongoDB.

Row count of every table after the live sync — MySQL 8.0 · gcode

```
mysql> SELECT 'platform' AS table_name, COUNT(*) AS row_count FROM platform UNION ALL
SELECT 'country', COUNT(*) FROM country UNION ALL SELECT 'city', COUNT(*) FROM city UNION ALL
SELECT 'city_alias', COUNT(*) FROM city_alias UNION ALL SELECT 'app_user', COUNT(*) FROM app_user UNION ALL
SELECT 'platform_account', COUNT(*) FROM platform_account UNION ALL SELECT 'contest', COUNT(*) FROM contest UNION ALL
SELECT 'problem', COUNT(*) FROM problem UNION ALL SELECT 'tag', COUNT(*) FROM tag UNION ALL
SELECT 'problem_tag', COUNT(*) FROM problem_tag UNION ALL SELECT 'verdict', COUNT(*) FROM verdict UNION ALL
SELECT 'submission', COUNT(*) FROM submission UNION ALL SELECT 'city_daily_stats', COUNT(*) FROM city_daily_stats UNION ALL
SELECT 'city_follow', COUNT(*) FROM city_follow UNION ALL SELECT 'ingest_outbox', COUNT(*) FROM ingest_outbox UNION ALL
SELECT 'sync_run', COUNT(*) FROM sync_run UNION ALL SELECT 'geocode_cache', COUNT(*) FROM geocode_cache;
```

table_name	row_count
platform	1
country	47
city	153
city_alias	153
app_user	2
platform_account	1140
contest	2196
problem	11603
tag	38
problem_tag	33078
verdict	17
submission	1708
city_daily_stats	177
city_follow	0
ingest_outbox	1637
sync_run	79
geocode_cache	42

17 rows in set

Figure 45: Row count of every table after the live sync

submission (latest rows written by the live poller) — MySQL 8.0 · gcode

```
mysql> SELECT s.submission_id, s.external_id, a.handle, p.external_key AS problem, v.code AS verdict, s.language, s.submitted_at
FROM submission s
JOIN platform_account a ON a.account_id = s.account_id
JOIN problem p ON p.problem_id = s.problem_id
JOIN verdict v ON v.verdict_id = s.verdict_id
ORDER BY s.submission_id DESC LIMIT 8;
```

submission_id	external_id	handle	problem	verdict	language	submitted_at
2899	398035413	Vingying0	877D	TESTING	C++23 (GCC 14-64, msys2)	2026-11-10 13:12:07
2898	398035414	XiangXunyi	1780E	TESTING	C++23 (GCC 14-64, msys2)	2026-11-10 13:12:08
2897	398035416	2026030202025	1795A	TESTING	GNU C11	2026-11-10 13:12:09
2896	398035418	sandbad	2279A	TESTING	GNU C11	2026-11-10 13:12:11
2895	398035420	lujain_sh	1856C	TESTING	C++23 (GCC 14-64, msys2)	2026-11-10 13:12:12
2894	398035421	crj0x	1992C	TESTING	C++23 (GCC 14-64, msys2)	2026-11-10 13:12:12
2893	398035424	anukar_sh	4A	RUNTIME_ERROR	Python 3	2026-11-10 13:12:13
2892	398035425	Player__	2261A	TESTING	C++23 (GCC 14-64, msys2)	2026-11-10 13:12:14

8 rows in set

Figure 46: submission (latest rows written by the live poller)

platform_account (location resolved through city_alias) — MySQL 8.0 · gcode

```
mysql> SELECT account_id, handle, raw_city, raw_country, city_id, country_id, location_status, rating
FROM platform_account
WHERE location_status = 'CITY_RESOLVED'
ORDER BY rating DESC LIMIT 8;
```

account_id	handle	raw_city	raw_country	city_id	country_id	location_status	rating
185	Mathematical_analogy	New Delhi	India	44	7	CITY_RESOLVED	2341
425	Aldk	Atbasar	Kazakhstan	80	14	CITY_RESOLVED	2325
956	ender_shayan	Tehran	Iran	113	17	CITY_RESOLVED	2324
1123	msab3f	Tehran	Iran	113	17	CITY_RESOLVED	2311
68	Tong_Yuxin	Changsha	China	17	1	CITY_RESOLVED	2310
131	yaufung	Hong Kong	Hong Kong	32	18	CITY_RESOLVED	2252
137	surgutti	Wrocław	Poland	34	19	CITY_RESOLVED	2247
839	lagrange_i_f	Nanning	China	142	1	CITY_RESOLVED	2234

8 rows in set

Figure 47: platform_account (location resolved through city_alias)

city + country — MySQL 8.0 · gcode

```
mysql> SELECT c.city_id, c.name AS city, co.name AS country, c.latitude, c.longitude
FROM city c JOIN country co ON co.country_id = c.country_id
ORDER BY c.city_id LIMIT 8;
```

city_id	city	country	latitude	longitude
1	Huizhou	China	23.112515	114.412701
2	Assiut	Egypt	27.046304	31.014803
3	Riga	Latvia	56.949398	24.105185
4	Khorugh	Tajikistan	37.490917	71.548922
5	Aleppo	Syria	36.199240	37.163725
6	Saint Petersburg	Russia	59.960674	30.158655
7	Hyderabad	India	17.360589	78.474061
8	Delhi	India	28.632803	77.219771

8 rows in set

Figure 48: city + country

problem + tags (problem_tag) — MySQL 8.0 · gcode

```
mysql> SELECT p.problem_id, p.external_key, p.name, p.rating,
       GROUP_CONCAT(t.name ORDER BY t.name SEPARATOR ', ') AS tags
FROM problem p
LEFT JOIN problem_tag pt ON pt.problem_id = p.problem_id
LEFT JOIN tag t ON t.tag_id = pt.tag_id
GROUP BY p.problem_id ORDER BY p.problem_id LIMIT 8;
```

problem_id	external_key	name	rating	tags
1	2284F	Fun Times	NULL	combinatorics, data structures, dp, math, matrices
2	2284E	My Mind Is Going	NULL	binary search, dfs and similar, graphs, interactive
3	2284D	The Theorem of Two Cans	NULL	communication, constructive algorithms, dfs and similar, graphs, interactive, math, trees
4	2284C2	Disgusting + GCD (Checker Version)	NULL	brute force, dp, graphs, greedy, math, number theory
5	2284C1	Disgusting + GCD (Constructive Version)	NULL	constructive algorithms, dp, graphs, greedy, math, number theory
6	2284B	Introduction to the Niche Internet	NULL	constructive algorithms, greedy, sortings
7	2284A2	15K4JR3B (Hard Version)	NULL	greedy, math, number theory
8	2284A1	15K4JR3B (Easy Version)	NULL	greedy, math, number theory

8 rows in set

Figure 49: problem + tags (problem_tag)

verdict (lookup table, includes TESTING) — MySQL 8.0 · gcode

```
mysql> SELECT verdict_id, code, is_accepted FROM verdict ORDER BY verdict_id;
```

verdict_id	code	is_accepted
1	OK	1
2	WRONG_ANSWER	0
3	TIME_LIMIT_EXCEEDED	0
4	MEMORY_LIMIT_EXCEEDED	0
5	RUNTIME_ERROR	0
6	COMPILATION_ERROR	0
7	IDLENESS_LIMIT_EXCEEDED	0
8	PRESENTATION_ERROR	0
9	CHALLENGED	0
10	SKIPPED	0
11	PARTIAL	0
12	REJECTED	0
13	FAILED	0
14	CRASHED	0
15	SECURITY_VIOLATED	0
16	INPUT_PREPARATION_CRASHED	0
17	TESTING	0

17 rows in set

Figure 50: verdict (lookup table, includes TESTING)

city_daily_stats (kept current by triggers) — MySQL 8.0 · gcode

```
mysql> SELECT d.city_id, c.name AS city, d.stat_date, d.total_submissions, d.accepted_submissions
FROM city_daily_stats d JOIN city c ON c.city_id = d.city_id
ORDER BY d.stat_date DESC, d.total_submissions DESC LIMIT 8;
```

city_id	city	stat_date	total_submissions	accepted_submissions
113	Tehran	2026-11-10	7	4
16	Pune	2026-11-10	6	5
44	New Delhi	2026-11-10	6	1
82	Dhaka	2026-11-10	6	3
103	Lucknow	2026-11-10	6	2
124	Aqtöbe	2026-11-10	6	0
18	Ho Chi Minh City	2026-11-10	5	2
22	Damascus	2026-11-10	5	3

8 rows in set

Figure 51: city_daily_stats (kept current by triggers)

app_user (passwords stored only as bcrypt hashes) — MySQL 8.0 · gcode

```
mysql> SELECT user_id, email, LEFT(password_hash, 29) AS password_hash, role, created_at FROM app_user;
```

user_id	email	password_hash	role	created_at
1	admin@gcode.test	$2b$10$IX5fFl6NIbhKgwPTIHbEe.	ADMIN	2026-11-10 18:36:04
2	viewer@gcode.test	$2b$10$gSDh0pjdA8Dq2dVC6XUeye	USER	2026-11-10 18:36:04

2 rows in set

Figure 52: app_user (passwords stored only as bcrypt hashes)

ingest_outbox (raw JSON waiting for / copied to MongoDB) — MySQL 8.0 · gcode

```
mysql> SELECT outbox_id, kind, external_key,
       LEFT(JSON_UNQUOTE(JSON_EXTRACT(payload, '$.verdict')), 20) AS payload_verdict,
       created_at, processed_at
FROM ingest_outbox WHERE kind = 'submission'
ORDER BY outbox_id DESC LIMIT 6;
```

outbox_id	kind	external_key	payload_verdict	created_at	processed_at
1637	submission	398035369	WRONG_ANSWER	2026-11-10 13:12:19	NULL
1636	submission	398035371	OK	2026-11-10 13:12:19	NULL
1635	submission	398035375	OK	2026-11-10 13:12:19	NULL
1634	submission	398035379	OK	2026-11-10 13:12:19	NULL
1633	submission	398035401	OK	2026-11-10 13:12:19	NULL
1632	submission	398035413	TESTING	2026-11-10 13:12:19	NULL

6 rows in set

Figure 53: ingest_outbox (raw JSON waiting for / copied to MongoDB)

sync_run (job log shown on the admin dashboard) — MySQL 8.0 · gcode

```
mysql> SELECT run_id, job, started_at, status, fetched, inserted, updated, message
FROM sync_run ORDER BY run_id DESC LIMIT 8;
```

run_id	job	started_at	status	fetched	inserted	updated	message
79	poll	2026-11-10 13:12:16	OK	300	11	5	7 new accounts
78	mongo	2026-11-10 13:12:11	OK	42	42	0	NULL
77	poll	2026-11-10 13:12:05	OK	300	6	5	NULL
76	poll	2026-11-10 13:11:58	OK	300	15	10	6 new accounts
75	mongo	2026-11-10 13:11:56	OK	44	44	0	NULL
74	poll	2026-11-10 13:11:50	OK	300	9	5	7 new accounts
73	poll	2026-11-10 13:11:42	OK	300	10	6	6 new accounts
72	mongo	2026-11-10 13:11:41	OK	25	25	0	NULL

8 rows in set

Figure 54: sync_run (job log shown on the admin dashboard)

geocode_cache (each place name geocoded once) — MySQL 8.0 · gcode

```
mysql> SELECT raw_city, raw_country, city_id, attempted_at FROM geocode_cache ORDER BY attempted_at DESC LIMIT 6;
```

raw_city	raw_country	city_id	attempted_at
Akhalts'ikhe	Georgia	NULL	2026-11-10 13:12:19
Indore	India	153	2026-11-10 13:12:17
Zhongshan	China	152	2026-11-10 13:12:15
Weifang	China	151	2026-11-10 13:12:14
Hassan	India	150	2026-11-10 13:12:12
Mazyr	Belarus	149	2026-11-10 13:12:07

6 rows in set

Figure 55: geocode_cache (each place name geocoded once)

city_analytics — top cities (one document per city) MongoDB 8.0 · gcode

```
gcode> db.city_analytics.find({}, { _id: 0, cityId: 1, city: 1, country: 1, "totals.submissions": 1, "totals.users": 1,
"totals.acceptedPct": 1 })
  .sort({ "totals.submissions": -1 }).limit(6)
```


```
[
  {
    "cityId": 16,
    "city": "Pune",
    "country": "India",
    "totals": {
      "submissions": 19,
      "users": 6,
      "acceptedPct": 32
    }
  },
  {
    "cityId": 8,
    "city": "Delhi",
    "country": "India",
    "totals": {
      "submissions": 14,
      "users": 8,
      "acceptedPct": 43
    }
  },
  {
    "cityId": 33,
    "city": "Cairo",
    "country": "Egypt",
    "totals": {
      "submissions": 14,
      "users": 11,
      "acceptedPct": 71
    }
  },
  {
    "cityId": 18,
    "city": "Ho Chi Minh City",
    "country": "Vietnam",
    "totals": {
      "submissions": 14,
      "users": 6,
      "acceptedPct": 43
    }
  },
  {
    "cityId": 82,
    "city": "Dhaka",
    "country": "Bangladesh",
    "totals": {
      "submissions": 12,
      "users": 8,
      "acceptedPct": 42
    }
  },
  {
    "cityId": 32,
    "city": "Hong Kong",
    "country": "Hong Kong",
    "totals": {
      "submissions": 10,
      "users": 2,
      "acceptedPct": 10
    }
  }
]
```


Figure 56: city_analytics — top cities (one document per city)

Next.js-Project Tree Structure

```
gcode/
├── app/
│   ├── admin/
│   │   ├── page.js    # admin dashboard
│   ├── api/
│   │   ├── admin/
│   │   │   ├── stats/
│   │   │   │   ├── route.js    # GET dashboard data (admin)
│   │   │   ├── sync/
│   │   │   │   ├── route.js    # POST rebuild analytics (admin)
│   │   ├── auth/
│   │   │   ├── login/
│   │   │   │   ├── route.js    # POST login (bcrypt + cookie)
│   │   │   ├── logout/
│   │   │   │   ├── route.js    # POST logout
│   │   │   ├── me/
│   │   │   │   ├── route.js    # GET current user
│   │   ├── problems/
│   │   │   ├── route.js    # GET problems (MySQL, FULLTEXT, CTE)
│   │   ├── regions/
│   │   │   ├── [id]/
│   │   │   │   ├── route.js    # GET one city panel (MongoDB)
│   │   │   ├── route.js    # GET all city dots (MongoDB)
│   │   ├── tags/
│   │   │   ├── route.js    # GET tags (MySQL)
│   ├── globals.css    # styles
│   ├── layout.js    # root layout + navbar
│   ├── login/
│   │   ├── page.js    # login form
│   ├── page.js    # globe page
│   ├── problems/
│   │   ├── page.js    # problems table
├── components/
│   ├── GlobeView.js    # 3D globe (react-globe.gl)
│   ├── Navbar.js    # navigation + login state
│   ├── RegionPanel.js    # city side panel
├── db/
│   ├── migrations/
│   │   ├── 002_live_sync.sql    # DDL: live-sync tables, trigger, procedure
│   │   ├── 003_roles.sql    # MySQL roles + grants
│   ├── schema.sql    # DDL: tables, triggers, procedure, views
├── lib/
│   ├── api.js    # API helpers
│   ├── auth.js    # client login helpers
│   ├── db.js
│   ├── db/
│   │   ├── mongodb.js    # MongoDB client
│   │   ├── mysql.js    # MySQL pools (root / viewer / admin)
│   ├── session.js    # signed session cookie, requireRole()
├── scripts/
│   ├── build-city-analytics.cjs    # MongoDB aggregation → city_analytics
│   ├── codeforces/
│   │   ├── api.js    # rate-limited Codeforces client
│   │   ├── collect.js
│   │   ├── geocode-cities.js
│   │   ├── inspect.mjs
│   │   ├── normalize.js
│   ├── env.js
│   ├── geo_mapper.js
│   ├── import.js
│   ├── import_mysql.js    # initial snapshot → MySQL
│   ├── sync/
│   │   ├── backfill-mongo.js
│   │   ├── geocode-pending.js    # new cities → coordinates
│   │   ├── outbox-to-mongo.js    # outbox → MongoDB (upsert)
│   │   ├── poll-submissions.js    # Codeforces → MySQL + outbox
│   │   ├── run-job.js
```

```
│   │   ├── worker.js     # runs every job on a timer
│   ├── test-verdict-trigger.js     # trigger test (rolls back)
├── .env.example
├── .gitignore
├── README.md
├── eslint.config.mjs
├── jsconfig.json
├── next.config.mjs
├── package-lock.json
├── package.json
├── data/codeforces/{raw,normalized}/     # collected Codeforces snapshot (JSON)
└── public/textures/                      # globe textures
```

Sample Screenshots of Application Code and Demo

Application code

Login: bcrypt check against app_user, then a signed httpOnly cookie — app/api/auth/login/route.js

```
  const email = String(body?.email ?? "").trim();
  const password = String(body?.password ?? "");
  if (!email || !password) {
    return Response.json({ error: "Email and password are required" }, { status: 400 });
  }

  try {
    const [rows] = await getPool().query(
      "SELECT user_id, email, password_hash, role FROM app_user WHERE email = ? LIMIT 1",
      [email]
    );
    const row = rows[0];
    const valid = row ? await bcrypt.compare(password, row.password_hash) : false;
    if (!valid) {
      return Response.json({ error: "Invalid email or password" }, { status: 401 });
    }

    const user = { id: row.user_id, email: row.email, username: row.email, role: roleFromDb(row.role) };
    await setSessionCookie(user);
    return Response.json({ username: user.username, email: user.email, role: user.role });
  } catch (error) {
    return dbError(error, "login");
  }
}
```

Figure 57: Login route: bcrypt comparison against app_user and a signed httpOnly session cookie

Role check used by every admin API route — lib/session.js

```
// Use at the top of a protected API route:
//   const { error } = await requireRole("admin");
//   if (error) return error;
export async function requireRole(role) {
  const session = await getSession();
  if (!session) return { error: Response.json({ error: "Please log in." }, { status: 401 }) };
  if (session.role !== role) return { error: Response.json({ error: "Not allowed for your role." }, { status: 403 }) };
  return { session };

```

Figure 58: requireRole(): every admin API route checks the role on the server (401 / 403)

Public routes connect as the read-only MySQL role, admin routes as gcode_admin — lib/db/mysql.js

```
// Role-based database access (db/migrations/003_roles.sql).
// Public read-only routes connect as gcode_viewer (SELECT only); the admin routes connect
// as gcode_admin. If DB_VIEWER_USER / DB_ADMIN_USER are not set, both fall back to getPool().
export function getViewerPool() {
  const user = process.env.DB_VIEWER_USER;
  if (!user) return getPool();
  return makePool("__gcodeViewerPool", { user, password: process.env.DB_VIEWER_PASSWORD ?? "" });
}

export function getAdminPool() {
  const user = process.env.DB_ADMIN_USER;
  if (!user) return getPool();
  return makePool("__gcodeAdminPool", { user, password: process.env.DB_ADMIN_PASSWORD ?? "" });
}
```

Figure 59: Public routes connect as the read-only MySQL role, admin routes as gcode_admin

City panel API: reads the precomputed MongoDB document — app/api/regions/[id]/route.js

```
// GET /api/regions/:id
// The side panel for one city: the whole precomputed city_analytics document (MongoDB).

import { getMongoDb } from "@/lib/db/mongodb";
import { dbError, toRegion } from "@/lib/api";

export const dynamic = "force-dynamic";

export async function GET(request, { params }) {
  const { id } = await params;
  const cityId = Number(id);
  if (!Number.isInteger(cityId)) {
    return Response.json({ error: "Invalid city id" }, { status: 400 });
  }

  try {
    const db = await getMongoDb();
    const doc = await db.collection("city_analytics").findOne({ cityId }, { projection: { _id: 0 } });
    if (!doc) {
      return Response.json({ error: "City not found" }, { status: 404 });
    }

    return Response.json({
      ...toRegion(doc),
      topTags: (doc.topics ?? []).map((t) => ({ name: t.tag, count: t.count })),
      verdicts: (doc.verdicts ?? []).map((v) => ({ name: v.code, count: v.count })),
      languages: doc.languages ?? [],
      difficulty: (doc.difficulty ?? []).map((d) => ({ name: d.band, count: d.count })),
      topProblems: (doc.mostSolved ?? []).map((p) => ({
        id: p.key, name: p.name, rating: p.rating ?? null, solvedCount: p.solvers,
      })),
    });
  } catch (error) {
    return dbError(error, "city");
  }
}
```

Figure 60: City panel API: one indexed read of the precomputed MongoDB document

Live sync: idempotent upsert of each Codeforces submission — scripts/sync/poll-submissions.js

```
        // affectedRows: 1 = inserted, 2 = existing row changed, 0 = identical row
        const [result] = await db.execute(
          `INSERT INTO submission
             (platform_id, external_id, account_id, problem_id, verdict_id, language, submitted_at)
           VALUES (?, ?, ?, ?, ?, ?, ?)
           ON DUPLICATE KEY UPDATE verdict_id = VALUES(verdict_id)`,
          [platformId, s.id, accountId, problemId, verdictId, norm.language?.slice(0, 50) ?? null,
           toMySQLDate(norm.submittedAt)]
        );

        if (result.affectedRows === 1) inserted++;
        if (result.affectedRows === 2) updated++;
        if (result.affectedRows > 0) await outbox(db, "submission", s.id, s);
      }

```

Figure 61: Live sync: idempotent upsert of each Codeforces submission

MongoDB aggregation that builds each city's 'top topics' — scripts/build-city-analytics.cjs

```
/** "top topics": a submission counts once per tag of its problem (same rule as v_city_tag_counts) */
function topicsPipeline() {
  return [
    { $unwind: '$tags' },
    { $group: { _id: { cityId: '$cityId', tag: '$tags' }, count: { $sum: 1 } } },
    { $sort: { count: -1, '_id.tag': 1 } },
    { $group: { _id: '$_id.cityId', topics: { $push: { tag: '$_id.tag', count: '$count' } } } },
    { $project: { _id: 0, cityId: '$_id', topics: { $slice: ['$topics', TOP.topics] } } },
    mergeInto('discard'),
  ];
}
```

Figure 62: MongoDB aggregation pipeline for a city's top topics

Application demo



Figure 63: Globe page: every green dot is a city with Codeforces activity (real data)



Figure 64: Clicking a city (Pune) opens its live analytics panel

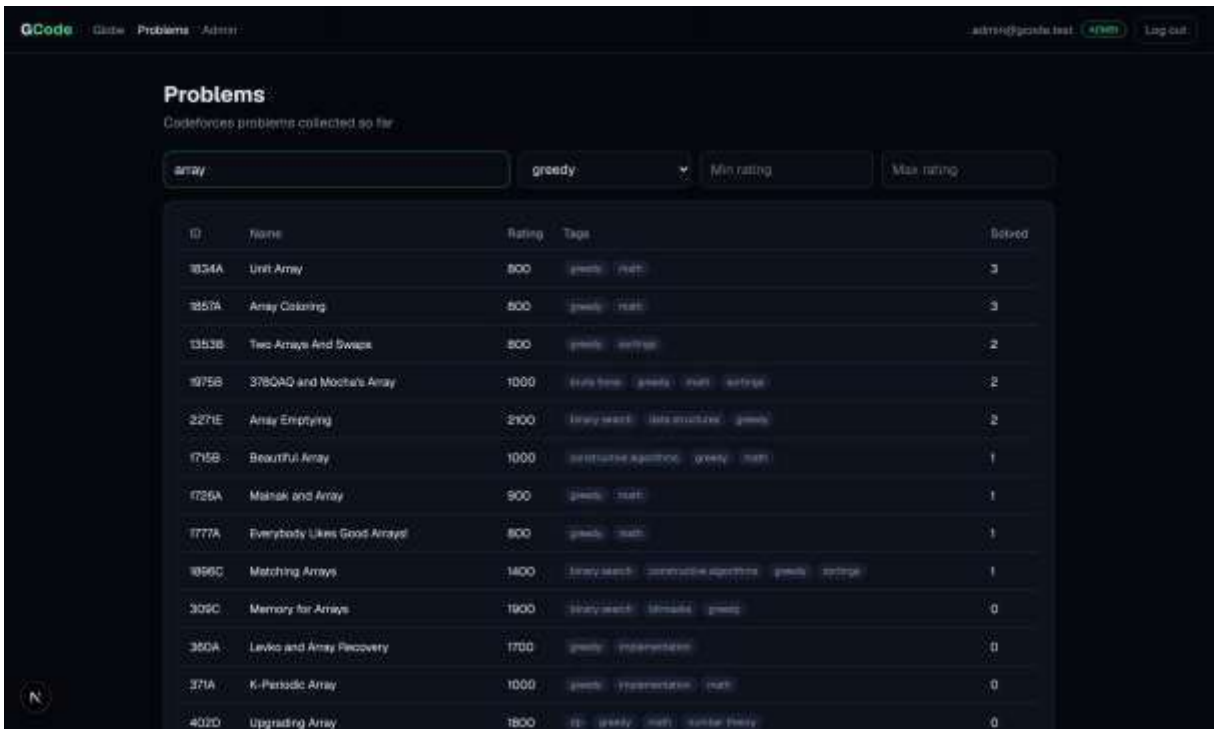


Figure 65: Problems page: full-text search 'array' filtered by the tag 'greedy', sorted by solvers

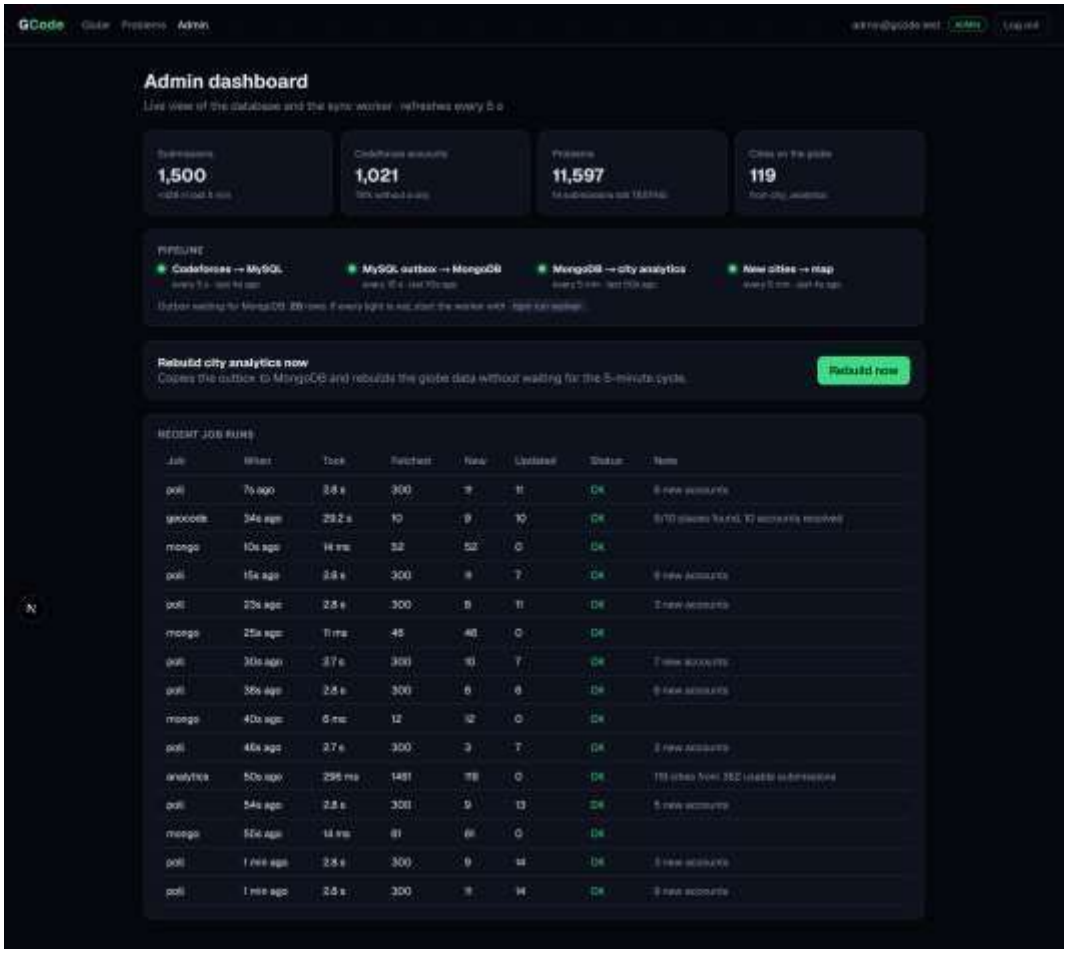


Figure 66: Admin dashboard: live totals, pipeline health lights and the job log

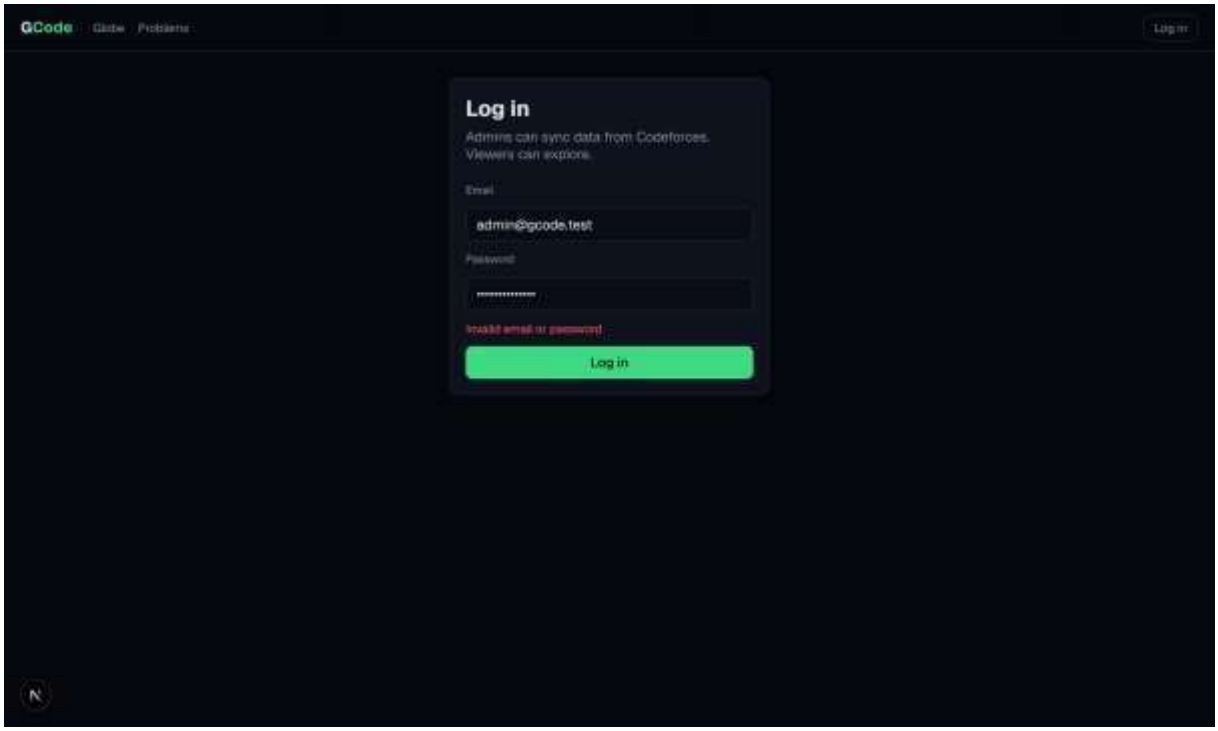


Figure 67: Login rejects a wrong password

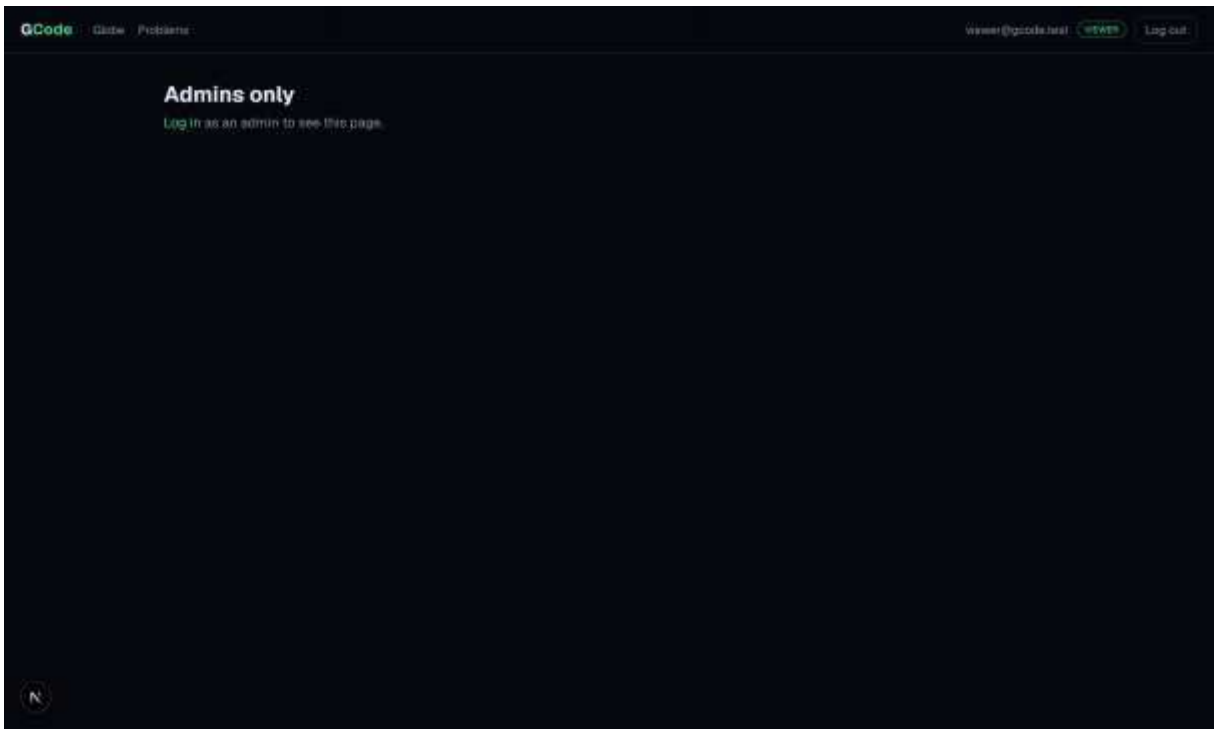


Figure 68: A Viewer opening /admin is refused



Figure 69: A Viewer calling the admin API directly gets HTTP 403 from the server

Additional Features Beyond the Requirements

Besides the features the rubric requires, GCode uses the following database and engineering features. Each one is implemented in the repository and demonstrated with real output earlier in this report.

Feature	Where it is implemented	Shown in
Full-text search: FULLTEXT index, MATCH … AGAINST in BOOLEAN MODE	db/schema.sql (ft_problem_name), app/api/problems/route.js	SQL DML: full-text search; Problems page screenshot
Common table expressions: CTE and recursive CTE	app/api/problems/route.js (most-solved CTE), rating-ladder query	SQL DML: CTE section
Transactions: START TRANSACTION / SAVEPOINT / COMMIT / ROLLBACK	sp_rebuild_city_stats (EXIT HANDLER → ROLLBACK, RESIGNAL); poller, geocoder and importer use beginTransaction / commit / rollback	SQL DML: transactions
GRANT and REVOKE, MySQL roles, default roles	db/migrations/003_roles.sql; lib/db/mysql.js uses a viewer pool and an admin pool	SQL DML: role-based access, GRANT and REVOKE
Triggers (3), stored procedures (2, one with an OUT parameter), views (2)	db/schema.sql, db/migrations/002_live_sync.sql	SQL DDL; SQL DML: triggers and procedures
Generated column, JSON column, composite foreign key, CHECK constraints	city_alias.country_key, ingest_outbox.payload, platform_account (city_id, country_id) → city	SQL DDL; CHECK / FK error figures
Idempotent upserts: no duplicate rows ever	INSERT … ON DUPLICATE KEY UPDATE (MySQL); bulkWrite upserts and unique indexes (MongoDB)	SQL DML: duplicate protection; NoSQL: unique indexes
MongoDB aggregation pipelines: $lookup, $unwind, $group, $project, $merge, $out	scripts/build-city-analytics.cjs; atomic renameCollection swap of the finished collection	NoSQL: aggregation
MongoDB $jsonSchema validators and unique indexes	city_analytics and saved_views collections	NoSQL: validator rejecting a bad document
Transactional outbox pattern (MySQL → MongoDB)	ingest_outbox written in the same transaction as the data; scripts/sync/outbox-to-mongo.js	Results: ingest_outbox table
Live background worker with job log	scripts/sync/worker.js: poll every 5 s, MongoDB sync every 15 s, analytics and geocoding every 5 min, cleanup every hour; sync_run log	Results: sync_run; admin dashboard
Geocoding with a cache and rate limit	scripts/sync/geocode-pending.js, geocode_cache, city_alias	Results: geocode_cache
Secure authentication	bcrypt password hashes, HMAC-signed httpOnly session cookie, server-side requireRole() (HTTP 403)	Screenshots: login, viewer refused
Interactive 3D globe (WebGL)	components/GlobeView.js (react-globe.gl)	Screenshots: globe and city panel

Evaluation Rubrics- Completion Status

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	**Completed** (sections 3, 4)
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	**Completed** (sections 2, 5, 7)
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	**Completed** (section 6)
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	**Completed** (sections 6, 7)
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	**Completed** (sections 9, 10)
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	**Completed** (sections 6, 10)
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	**Completed** (GitHub, this report)
	Total	All 7 criteria implemented	**Completed**

Coverage of each evaluation component

#	Evaluation component	Max	Where it is covered in this report	Status
1	ER diagram	2	E-R Model: entities, weak entity, relationships, cardinality, ER diagram	Done
2	Relational schema with keys, relationships and constraints	3	Relational Model: 17 relations, PK/FK, schema diagram	Done
3	SQL implementation: DDL with constraints	4	SQL DDL: PK, FK, UNIQUE, CHECK, NOT NULL, DEFAULT, FULLTEXT, generated column	Done
4	NoSQL database creation	4	NoSQL: collections, $jsonSchema validators, unique indexes	Done
5	Technology justification	1	User Requirements: technology stack table and why MongoDB	Done
6	DML operations	4	SQL DML: INSERT, UPDATE, DELETE, duplicate-safe upsert	Done
7	Nested / correlated / recursive and join queries	4	SQL DML: nested, correlated, recursive CTE, INNER/LEFT JOIN, HAVING	Done

8	Advanced SQL: procedure/function, trigger, CTE	6	SQL DML: procedures with OUT parameter, 3 triggers, CTE; also views, transactions, full-text	Done
9	NoSQL feature: RUD	7	NoSQL: create, read, update, delete, aggregation pipeline, validation error	Done
10	Next.js interface	3	Screenshots: globe, city panel, problems page, admin dashboard, login	Done
11	Backend/API and database integration	2	Project tree and code: API routes reading MySQL and MongoDB	Done
12	Authentication, two roles, role-based access in the database	4	Login with bcrypt; ADMIN and USER roles; MySQL roles with GRANT/REVOKE	Done
13	Documentation, Git/GitHub	2	This report; GitHub repository below	Done
14	Originality, individual contribution, viva	4	Individual Contribution table below	Done
	Total	50		

Individual Contribution

Member	SRN	Contribution
Mohammad Bilal A	PES1UG24AM162	Next.js frontend: 3D globe, city panel, problems page, admin dashboard; API route integration; live-sync pipeline (worker, outbox to MongoDB); ER and relational schema diagrams.
Jayanth K	PES1UG24AM123	Codeforces data collection and normalization; MySQL schema, constraints and initial import; MongoDB city-analytics pipelines; authentication, roles and MySQL role-based access.

GitHub repository: https://github.com/Bilal-biryani/Gcode